

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC
ĐỀ TÀI: XÂY DỰNG PIPELINE VẬN HÀNH MLOPS
CHO HỆ THỐNG BẢO MẬT MẠNG SDN TỰ ĐỘNG
DỰA TRÊN HỌC MÁY TĂNG CƯỜNG

Môn học: NT548.Q21 – Công nghệ DevOps và ứng dụng

Giảng viên hướng dẫn: ThS. Lê Anh Tuấn

Thực hiện bởi nhóm 04, bao gồm:

- | | | |
|-------------------------|----------|-------------|
| 1. Nguyễn Lê Hạ My | 23520964 | Trưởng nhóm |
| 2. Bùi Nguyễn Tường Vân | 23521768 | Thành viên |
| 3. Trần Ngọc Thuý Vy | 23521837 | Thành viên |

Thời gian thực hiện: 04/2026 - 05/2026

MỤC LỤC

MỤC LỤC	2
Chương I. TỔNG QUAN	5
1.1. Đặt vấn đề	5
1.2. Mục tiêu thực hiện	5
1.3. Phạm vi và giới hạn	6
Chương II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	7
2.1. DevOps và MLOps.....	7
2.1.1. MLOps và vòng đời mô hình học máy.....	7
2.1.2. So sánh DevOps truyền thống và MLOps.....	7
2.2. Học máy tăng cường trong bảo mật mạng	8
2.3. Tổng quan về mạng SDN (Software-Defined Networking).....	8
2.3.2. Bảo mật mạng trong môi trường SDN	10
2.4. Các công nghệ nền tảng.....	10
2.4.1. Docker và Kubernetes/K3s.....	10
2.4.2. Mlflow	11
2.4.3. Prometheus và Grafana.....	11
2.4.4. AlertManager	12
2.4.5. Minio	12
Chương III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	14
3.1. Phân tích yêu cầu.....	14
3.1.1. Yêu cầu chức năng.....	14
3.1.2. Yêu cầu phi chức năng	14
3.2. Kiến trúc tổng thể hệ thống MLOps.....	15
3.2.1. Sơ đồ kiến trúc toàn hệ thống.....	15
3.2.2. Các thành phần và vai trò	16
3.2.3. Luồng dữ liệu và luồng điều khiển.....	17
3.2.4. Nguyên lý hoạt động của hệ thống.....	17
3.3. Thiết kế môi trường huấn luyện	18
3.3.1. Môi trường offline	18
3.3.2. Môi trường Online với ONOS Controller	19

3.4. Thiết kế pipeline MLOps end-to-end	19
Chương IV. TRIỂN KHAI HỆ THỐNG	20
4.1. Container hóa các thành phần.....	20
4.1.1. Dockerfile và cấu hình image.....	20
4.1.2. Triển khai trên Kubernetes/K3s.....	21
4.1.3. Quản lý biến môi trường và secrets.....	27
4.2. CI/CD Pipeline	27
4.2.1. Thiết kế pipeline build, test, deploy	27
4.2.2. Smoke test trong chế độ CI	28
4.2.3. Tự động kiểm tra chất lượng model trước khi deploy.....	28
4.2.4. Github Actions CI/CD	30
4.2.5. Triển khai CD nội bộ bằng deploy.sh	31
Để khắc phục hạn chế về bảo mật của GitHub-hosted runner, nhóm triển khai quy trình CD nội bộ (Local CD) thông qua script deploy.sh chạy trực tiếp trên máy ảo (VM):	31
4.3. Quản lý vòng đời mô hình với Mlflow.....	32
4.3.1. Experiment Tracking	32
4.3.2. Model Registry	34
4.3.3. Artifact Storage với MinIO.....	34
4.3.4. Cơ chế load model Production/Staging/Fallback	35
4.4. Hệ thống serving và control loop	36
4.4.1. Model serving endpoint.....	36
4.4.2. Control loop tích hợp RL agent với ONOS Controller	36
4.4.3. Chiến lược promote và rollback model	37
4.5. Giám sát và tự động vận hành	37
4.5.1. Prometheus metrics	37
4.5.2. Grafana dashboard.....	38
4.5.3. AlertManager	39
4.5.4. Auto-trigger webhook.....	39
Chương V. THỰC NGHIỆM VÀ ĐÁNH GIÁ	40
5.1. Môi trường thực nghiệm.....	40
5.2. Đánh giá pipeline CI/CD và MLOps.....	40

5.4. Đánh giá hệ thống giám sát	41
Chương VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	44
6.1. Tổng kết kết quả đạt được	44
6.2. Hạn chế của hệ thống hiện tại.....	44
6.3. Hướng phát triển.....	44
NGUỒN THAM KHẢO	45
LINK TÀI LIỆU	46

Chương I. TỔNG QUAN

1.1. Đặt vấn đề

Trong những năm gần đây, mạng khả lập trình (Software Defined Networking – SDN) ngày càng được ứng dụng rộng rãi nhờ khả năng quản lý tập trung, linh hoạt và dễ mở rộng. Tuy nhiên, kiến trúc tập trung của SDN cũng tạo ra nhiều nguy cơ bảo mật như tấn công DDoS, Packet-In Flood, Flow Table Overflow hoặc giả mạo lưu lượng mạng. Khi số lượng thiết bị và lưu lượng tăng cao, các phương pháp bảo mật truyền thống gặp nhiều hạn chế do phụ thuộc vào luật cố định và khó thích nghi với các hình thức tấn công mới.

Song song đó, học tăng cường (Reinforcement Learning – RL) đang trở thành một hướng tiếp cận tiềm năng trong lĩnh vực an ninh mạng nhờ khả năng học hành vi tối ưu dựa trên trạng thái môi trường và phản hồi reward. RL cho phép hệ thống tự động đưa ra quyết định như chặn luồng lưu lượng bất thường, giới hạn băng thông hoặc chuyển hướng lưu lượng độc hại nhằm giảm thiểu tác động của tấn công mạng trong môi trường SDN.

Tuy nhiên, việc triển khai một hệ thống RL thực tế cho SDN không chỉ yêu cầu mô hình học máy hoạt động hiệu quả mà còn đòi hỏi một kiến trúc DevOps–MLOps hoàn chỉnh nhằm quản lý toàn bộ vòng đời của mô hình. Các vấn đề như triển khai container, tự động hóa hạ tầng, CI/CD, monitoring, quản lý version mô hình và retrain tự động vẫn chưa được tích hợp đầy đủ trong nhiều nghiên cứu hiện nay.

Xuất phát từ thực tế đó, nhóm thực hiện đề tài xây dựng kiến trúc DevOps–MLOps tích hợp cho hệ thống bảo mật SDN sử dụng học tăng cường. Hệ thống hướng đến khả năng triển khai tự động, mở rộng linh hoạt, giám sát thời gian thực và hỗ trợ cập nhật mô hình RL liên tục mà không làm gián đoạn hoạt động của mạng SDN.

1.2. Mục tiêu thực hiện

Đề tài hướng đến việc xây dựng một kiến trúc DevOps–MLOps hoàn chỉnh cho hệ thống bảo mật SDN sử dụng học tăng cường với các mục tiêu chính như sau:

- Container hóa & Điều phối hạ tầng: Đóng gói toàn bộ thành phần (SDN Controller, RL Trainer/Inference, Monitoring) bằng Docker và vận hành trên Kubernetes để tối ưu khả năng mở rộng.

- Tự động hóa triển khai (CI/CD): Xây dựng luồng pipeline bằng GitHub Actions để tự động hóa quá trình build và deploy.
- Quản lý vòng đời AI (MLOps): Tích hợp MLflow nhằm theo dõi thí nghiệm, ghi nhận metric và quản lý phiên bản mô hình (Model Registry).
- Giám sát & Phản ứng tự động: Thiết lập Prometheus và Grafana để theo dõi hiệu năng toàn hệ thống; đồng thời xây dựng cơ chế tự động retrain và rollback mô hình dựa trên các chỉ số thực tế (reward, latency, stability).

1.3. Phạm vi và giới hạn

1.3.1. Phạm vi thực hiện

Trọng tâm của đề tài là xây dựng kiến trúc DevOps–MLOps cho bảo mật SDN trong môi trường mô phỏng, cụ thể bao gồm:

- Hạ tầng & mô hình: Mô phỏng mạng (ONOS, Mininet) và phát triển RL Agent ra quyết định phòng thủ.
- Triển khai & CI/CD: Đóng gói, điều phối hệ thống (Docker, Kubernetes) và tự động hóa pipeline (GitHub Actions).
- MLOps & Giám sát: Quản lý mô hình (MLflow), theo dõi hệ thống (Prometheus, Grafana) và thiết kế cơ chế retrain.

1.3.2. Giới hạn của đề tài

Bên cạnh các nội dung đạt được, đề tài vẫn còn một số giới hạn như sau:

- Môi trường: Triển khai hạ tầng (bao gồm cụm Kubernetes) ở quy mô thử nghiệm/mô phỏng, chưa áp dụng trên mạng SDN thực tế hay chịu tải cực lớn.
- Khả năng phòng thủ: RL Agent chỉ mới tập trung xử lý các nhóm tấn công phổ biến như DDoS và Flow Flooding.
- Mức độ tự động hóa: Cơ chế retrain hiện mới dừng ở mức bán tự động, hoạt động dựa trên các ngưỡng (threshold) giám sát.

Chương II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. DevOps và MLOps

DevOps là phương pháp kết hợp giữa phát triển phần mềm và vận hành hệ thống nhằm tự động hóa quá trình build, test và deploy ứng dụng. Trong khi đó, MLOps mở rộng DevOps cho các hệ thống machine learning bằng cách quản lý toàn bộ vòng đời của mô hình AI từ huấn luyện đến triển khai và monitoring.

Ứng dụng trong hệ thống:

- CI/CD: Sử dụng Github Actions để tự động kiểm thử và build Docker image. Kubernetes/K3s chịu trách nhiệm triển khai và quản lý các container (RL Serving, MLflow, Prometheus, v.v.).
- MLOps: Tích hợp MLflow để lưu trữ hyperparameter, model artifact và metric. Prometheus và Grafana liên tục giám sát hiệu năng mô hình sau khi deploy, hỗ trợ cơ chế cập nhật tự động mà không làm gián đoạn mạng.

2.1.1. MLOps và vòng đời mô hình học máy

MLOps giúp quản lý toàn bộ vòng đời của các mô hình học máy bao gồm thu thập dữ liệu, huấn luyện, tracking experiment, deployment, monitoring và retraining.

Trong đề tài này, MLflow được sử dụng để lưu hyperparameter, reward, metric và model artifact của từng lần huấn luyện. Sau khi model được deploy, Prometheus và Grafana sẽ tiếp tục theo dõi hiệu năng của hệ thống nhằm phát hiện tình trạng giảm chất lượng hoặc bất ổn của mô hình.

Việc áp dụng MLOps giúp hệ thống RL hoạt động ổn định hơn và hỗ trợ cập nhật model liên tục mà không làm gián đoạn mạng SDN.

2.1.2. So sánh DevOps truyền thống và MLOps

DevOps truyền thống tập trung chủ yếu vào quản lý source code và triển khai ứng dụng, trong khi MLOps phải quản lý thêm dữ liệu, model artifact, experiment và model version. Ngoài việc monitoring hệ thống, MLOps còn yêu cầu theo dõi hiệu năng của mô hình AI như accuracy, reward hoặc latency.

Do đó, MLOps phức tạp hơn DevOps truyền thống vì phải đảm bảo cả application và AI model hoạt động ổn định trong môi trường production.

2.2. Học máy tăng cường trong bảo mật mạng

Học máy tăng cường bao gồm các thành phần chính như agent, environment, state, action và reward. Agent là tác nhân đưa ra quyết định, environment là môi trường mà agent tương tác, state biểu diễn trạng thái hiện tại của hệ thống, action là hành động mà agent lựa chọn và reward là phản hồi từ môi trường sau khi thực hiện action.

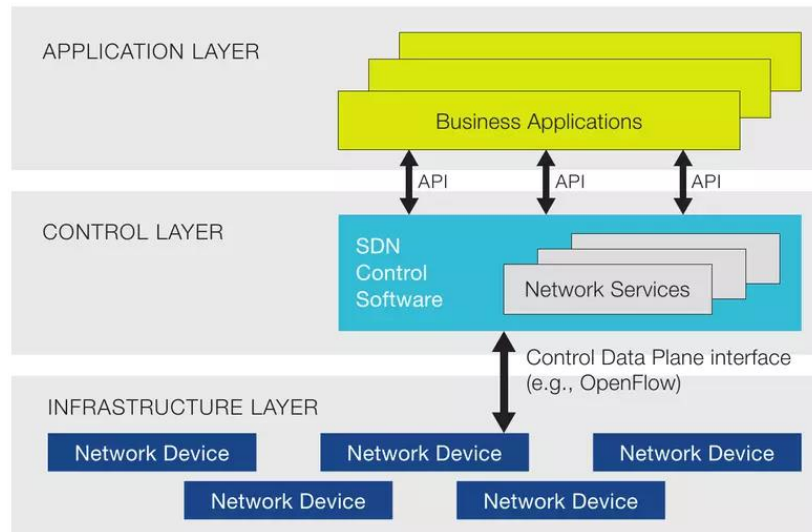
- Nguyên lý hoạt động: RL cho phép một tác nhân (agent) học cách ra quyết định tối ưu thông qua tương tác với môi trường dựa trên phản hồi (reward), không cần dữ liệu gán nhãn trước.
- Áp dụng vào bảo mật: Agent phân tích trạng thái mạng (như packet rate, latency) để đưa ra hành động phòng thủ (block flow, giới hạn băng thông). Nếu hành động chặn đúng tấn công và duy trì QoS, agent nhận reward cao; ngược lại, nếu ảnh hưởng đến traffic hợp lệ, reward sẽ bị giảm.
- Mô hình hóa MDP: Bài toán bảo mật được mô hình hóa dưới dạng Markov Decision Process (MDP). Mục tiêu của RL agent là tìm ra chính sách (policy) tối đa hóa tổng reward dài hạn, nhằm bảo vệ mạng một cách tối ưu.

2.3. Tổng quan về mạng SDN (Software-Defined Networking)

Software-Defined Networking (SDN) là mô hình mạng hiện đại cho phép tách biệt mặt phẳng điều khiển (Control Plane) và mặt phẳng chuyển tiếp dữ liệu (Data Plane). Trong kiến trúc truyền thống, các thiết bị mạng như router hoặc switch vừa thực hiện chức năng điều khiển vừa xử lý lưu lượng dữ liệu. Điều này làm cho việc quản lý mạng trở nên phức tạp và khó mở rộng.

SDN giải quyết vấn đề này bằng cách tập trung logic điều khiển tại SDN Controller, giúp quản trị viên có thể cấu hình, quản lý và điều khiển toàn bộ hệ thống mạng từ một điểm trung tâm. Các switch chỉ thực hiện nhiệm vụ forwarding dựa trên flow rule được controller gửi xuống thông qua giao thức như OpenFlow.

Nhờ khả năng lập trình linh hoạt, SDN được ứng dụng rộng rãi, trong đề tài này, SDN đóng vai trò là môi trường mạng trung tâm để triển khai hệ thống bảo mật sử dụng Reinforcement Learning.



Hình. Kiến trúc SDN

- Application Layer: Lớp ứng dụng chứa các ứng dụng quản lý mạng như load balancing, routing, monitoring và security application. Các ứng dụng ở lớp này giao tiếp với controller thông qua Northbound API để gửi yêu cầu điều khiển hoặc nhận thông tin trạng thái mạng.
- Control Layer: Là lớp điều khiển, nơi SDN Controller giữ vai trò trung tâm trong toàn bộ hệ thống. Controller chịu trách nhiệm quản lý topology mạng, thu thập trạng thái mạng, cài đặt flow rule và điều khiển hoạt động của các switch. Trong đề tài, nhóm sử dụng ONOS (Open Network Operating System) làm SDN Controller. ONOS là nền tảng mã nguồn mở hỗ trợ high availability, scalability, real-time network control và giao thức OpenFlow, phù hợp cho các hệ thống SDN có yêu cầu độ tin cậy cao.



- Infrastructure Layer: Bao gồm các switch OpenFlow và thiết bị mạng vật lý hoặc mô phỏng. Các switch thực hiện forwarding packet theo flow rule từ controller. Tầng này đóng vai trò nền tảng để triển khai, kiểm thử và đánh giá các chính sách phòng thủ mạng trong đề tài.

2.3.2. Bảo mật mạng trong môi trường SDN

Mặc dù SDN mang lại tính linh hoạt cao, kiến trúc tập trung của SDN cũng tạo ra nhiều nguy cơ bảo mật. Khi controller bị tấn công, toàn bộ mạng có thể bị ảnh hưởng.

Một số dạng tấn công phổ biến trong SDN gồm DDoS, Packet-In Flooding, Flow Table Overflow, IP Spoofing và Traffic Scanning. Các cuộc tấn công này thường tạo ra lượng lớn lưu lượng hoặc gói tin giả mạo nhằm làm quá tải controller và bảng flow của switch, dẫn đến tăng độ trễ, giảm hiệu năng xử lý và ảnh hưởng đến tính ổn định của hệ thống.

Do đó, việc áp dụng AI và Reinforcement Learning để tự động phát hiện và giảm thiểu tấn công mạng là hướng nghiên cứu tiềm năng trong các hệ thống SDN hiện đại.

2.4. Các công nghệ nền tảng

2.4.1. Docker và Kubernetes/K3s

Docker là nền tảng containerization cho phép đóng gói ứng dụng cùng toàn bộ dependency vào container. Trong đề tài, Docker được sử dụng để đóng gói các thành phần như RL Serving, Control Loop, Auto-trigger, MLflow, Prometheus, Grafana và AlertManager. Việc sử dụng Docker giúp giảm sai khác môi trường giữa quá trình phát triển, kiểm thử và triển khai.



Kubernetes/K3s được sử dụng để điều phối các container trong hệ thống. Thay vì chạy các service riêng lẻ thủ công, Kubernetes cho phép khai báo trạng thái mong muốn của hệ thống thông qua các manifest YAML. Khi một service bị lỗi, Kubernetes có thể tự động restart pod. Khi cần mở rộng, hệ thống có thể tăng số lượng replica hoặc dùng Horizontal Pod Autoscaler.



Trong đề tài, Kubernetes/K3s đảm nhiệm các chức năng chính:

Chức năng	Vai trò trong hệ thống
Quản lý container	Chạy ONOS, MLflow, MinIO, RL Serving, Control Loop, Prometheus, Grafana, AlertManager và Auto-trigger.
Quản lý cấu hình	Sử dụng ConfigMap cho Prometheus, Grafana và AlertManager.
Quản lý thông tin nhạy cảm	Sử dụng Secret cho MinIO access key và secret key.
Expose service	Sử dụng Service và NodePort để truy cập MLflow, Grafana, Prometheus, MinIO.
Tự động huấn luyện	Sử dụng CronJob/Job để chạy training DQN/PPO.
Tự động scale	Sử dụng HPA cho RL Serving DQN/PPO.
Tự phục hồi	Pod lỗi có thể được restart theo cơ chế của Deployment.

Như vậy, Docker cung cấp môi trường đóng gói ứng dụng, còn Kubernetes/K3s cung cấp môi trường vận hành cloud-native cho toàn bộ pipeline MLOps.

2.4.2. Mlflow

MLflow là nền tảng MLOps mã nguồn mở hỗ trợ quản lý vòng đời mô hình machine learning. Công cụ này giúp theo dõi experiment, lưu hyperparameter, metric và model artifact của từng lần huấn luyện.



Trong đề tài, MLflow được sử dụng để log reward, latency và các thông số của mô hình Reinforcement Learning. Model Registry hỗ trợ quản lý nhiều version model, giúp lựa chọn phiên bản tốt nhất để triển khai hoặc rollback khi cần thiết.

Ngoài ra, MLflow còn hỗ trợ đóng gói và triển khai mô hình dưới dạng REST API, giúp tích hợp RL model vào hệ thống SDN dễ dàng hơn.

2.4.3. Prometheus và Grafana

Prometheus là hệ thống monitoring mã nguồn mở dùng để thu thập và lưu trữ metric theo thời gian thực. Grafana là nền tảng visualization giúp hiển thị dashboard monitoring trực quan.



Trong đề tài, Prometheus được sử dụng để thu thập các metric như CPU, memory, latency, reward và inference time từ các service trong hệ thống. Grafana kết nối với Prometheus để hiển thị dashboard theo dõi trạng thái hoạt động của hệ thống và mô hình RL.

Các dashboard này hỗ trợ giám sát reward trend, action distribution, độ ổn định của hệ thống và hiệu năng của RL agent. Thông qua đó, nhóm có thể phát hiện sớm các bất thường và hỗ trợ cơ chế retrain hoặc rollback model khi cần thiết.

2.4.4. AlertManager

AlertManager là thành phần thuộc hệ sinh thái Prometheus, được sử dụng để quản lý và xử lý các cảnh báo phát sinh từ hệ thống monitoring. Khi Prometheus phát hiện metric vượt quá ngưỡng cho phép, AlertManager sẽ thực hiện phân loại, gom nhóm và định tuyến cảnh báo trước khi chuyển đến các kênh xử lý phù hợp.

Trong đề tài, AlertManager được tích hợp cùng Prometheus và Grafana để giám sát trạng thái hệ thống SDN và mô hình Reinforcement Learning. Công cụ này hỗ trợ phát hiện sớm các bất thường như lỗi dịch vụ, suy giảm hiệu năng mô hình hoặc các sự cố vận hành, từ đó nâng cao khả năng tự động hóa và đảm bảo tính ổn định của toàn bộ hệ thống.



2.4.5. Minio

MinIO là một hệ thống lưu trữ đối tượng mã nguồn mở, tương thích với chuẩn Amazon S3, được sử dụng để lưu trữ artifact trong pipeline MLOps. Trong đề tài, MinIO đóng vai trò là backend lưu trữ cho MLflow, cho phép quản lý tập trung các tệp mô hình, dữ liệu huấn luyện, log và kết quả đánh giá.



Vai trò của MinIO trong hệ thống được thể hiện như sau:

Vai trò	Mô tả
Lưu trữ Artifact	Lưu model, scaler, file log, file CSV kết quả huấn luyện và đánh giá
Backend cho Mlflow	Mlflow sử dụng MinIO như S3-compatible storage để lưu artifact
Hỗ trợ versioning model	Các artifact gắn với từng Mlflow run/model version được lưu lại có tổ chức
Hỗ trợ rollback	Khi cần rollback, hệ thống có thể tải lại model artifact của version cũ từ MinIO
Tách biệt tracking và storage	Mlflow quản lý metadata, còn MinIO lưu file artifact thực tế

Trong kiến trúc triển khai, MinIO được vận hành như một dịch vụ độc lập trên Kubernetes và kết nối với MLflow thông qua giao thức S3.

Luồng hoạt động cơ bản giữa Mlflow và MinIO:

Train DQN/PPO → MLflow log params/metrics → MLflow log artifacts → Artifact được lưu vào MinIO bucket → Model Registry quản lý version → RL Serving tải model/scaler để phục vụ inference

Như vậy, MinIO giúp pipeline MLOps có khả năng lưu trữ artifact ổn định, tách biệt và dễ mở rộng. Đây là thành phần quan trọng để hệ thống không chỉ huấn luyện model một lần mà còn có thể quản lý nhiều phiên bản mô hình trong suốt vòng đời vận hành.

Chương III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

3.1.1. Yêu cầu chức năng

Hệ thống phải thu thập được trạng thái mạng từ ONOS; xây dựng vector đặc trưng 8 chiều; huấn luyện mô hình RL (DQN, PPO); quản lý vòng đời bằng MLflow; cung cấp API Serving; điều phối tự động qua Control Loop và giám sát/cảnh báo bằng cụm Prometheus/Grafana/AlertManager.

Nhóm yêu cầu	Nội dung
Tính sẵn sàng	Các service quan trọng như ONOS, RL serving, Prometheus, Grafana cần tự restart khi lỗi.
Tính mở rộng	DQN và PPO được tách thành hai service riêng, có thể scale độc lập.
Tính quan sát	Hệ thống phải có metric cho reward, latency, packet loss, flow count, action và trạng thái model.
Tính tái lập	Các thành phần được container hóa để dễ chạy lại trên môi trường khác.
Tính bảo trì	Cấu hình service, port, endpoint và biến môi trường phải thống nhất.
Tính an toàn vận hành	Khi model mới kém hiệu quả, hệ thống cần hỗ trợ rollback hoặc fallback.

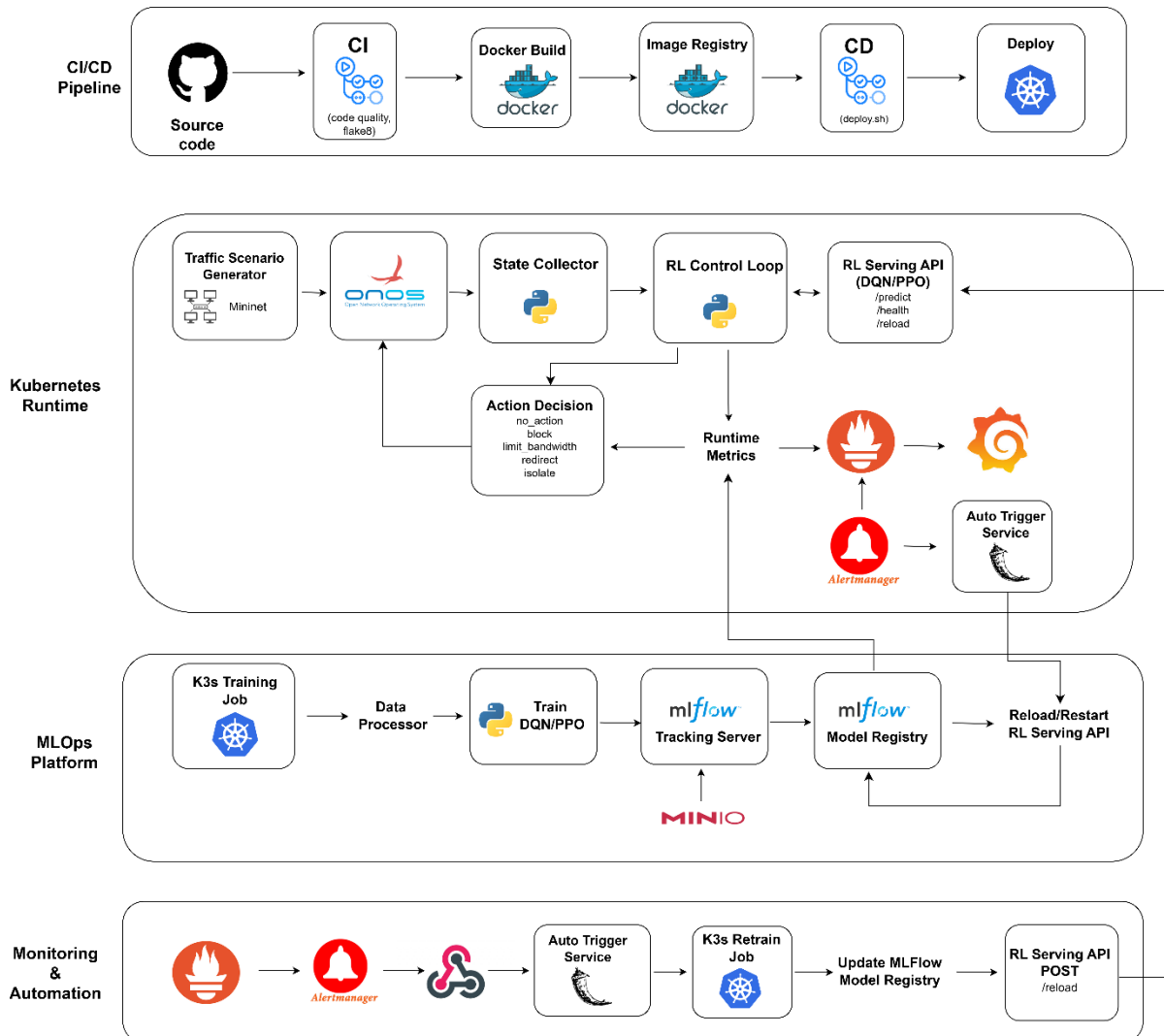
3.1.2. Yêu cầu phi chức năng

- Tính sẵn sàng & Tự phục hồi: Các service quan trọng phải tự restart khi gặp lỗi.
- Tính mở rộng: Tách biệt service cho DQN và PPO để scale độc lập.
- Tính quan sát: Tracking đầy đủ các metric (reward, latency, packet loss, v.v.).
- An toàn vận hành: Hỗ trợ rollback hoặc fallback về mô hình cũ khi mô hình mới kém hiệu quả.

3.2. Kiến trúc tổng thể hệ thống MLOps

3.2.1. Sơ đồ kiến trúc toàn hệ thống

- Luồng hoạt động chính: Mininet (sinh lưu lượng) → ONOS (quản lý) → State Collector (thu thập) → Control Loop → RL Serving (ra quyết định) → ONOS (áp dụng luật).
- Luồng MLOps & Giám sát: Song song với luồng chính, MLflow (cùng MinIO) quản lý tracking và versioning. Prometheus/Grafana giám sát metric thời gian thực, nếu có bất thường, AlertManager sẽ báo cho Auto-trigger để tự động retrain/promote.
- Tổ chức mã nguồn: Hệ thống được module hóa chặt chẽ theo các thư mục chức năng (control_loop, mlops, rl_engine, k8s, traffic_generator, v.v.) nhằm tối ưu cho việc container hóa và chạy CI/CD.



Hình. Sơ đồ kiến trúc hệ thống

Thư mục	Vai trò
.github/workflows	Chứa pipeline CI/CD chạy trên GitHub Actions.
control_loop	Chứa vòng lặp điều khiển runtime: lấy state, gọi model, tính reward, apply action về ONOS.
k8s	Chứa manifest Kubernetes/K3s để triển khai toàn bộ hệ thống.
mlops	Chứa auto-trigger webhook, promote/retrain logic và Dockerfile cho automation service.
prometheus	Chứa cấu hình scrape metrics và alert rules.
grafana	Chứa datasource và dashboard JSON.
alertmanager	Chứa cấu hình route alert đến auto-trigger webhook.
traffic_generator	Chứa topology Mininet và các hàm sinh traffic normal/attack.
models	Lưu model cục bộ dùng cho fallback hoặc test nhanh.
data	Lưu dữ liệu raw, processed, train, validation và test.

3.2.2. Các thành phần và vai trò

Thành phần	Vai trò
Mininet	Mô phỏng mạng SDN và sinh lưu lượng bình thường/tấn công.
ONOS Controller	Điều khiển mạng SDN, cung cấp REST API để lấy flow và cài rule.
State Collector	Thu thập trạng thái mạng từ ONOS.
State Builder	Chuẩn hóa trạng thái thành vector 8 chiều.
RL Agent	DQN/PPO chọn hành động phòng thủ
RL Serving	Cung cấp API /predict, /health, /reload
Control Loop	Vòng lặp lấy state, gọi model, tính reward và apply action.
Mlflow	Theo dõi training, log metric, lưu model và quản lý version
Prometheus	Thu thập metric runtime.
Grafana	Hiển thị dashboard quan sát hệ thống
AlertManager	Gửi cảnh báo đến auto-trigger

Auto Trigger	Tự động retrain/promote khi có cảnh báo
Github Actions	CI/CD kiểm tra code và build image

3.2.3. Luồng dữ liệu và luồng điều khiển

Luồng dữ liệu bắt đầu từ ONOS Controller. Control loop gọi REST API của ONOS để lấy danh sách flow, sau đó tính toán các đặc trưng như packet rate, byte rate, flow count, flow growth rate, entropy, latency, packet loss và controller CPU. Các đặc trưng này được đưa vào State Builder để tạo vector trạng thái 8 chiều.

Luồng điều khiển bắt đầu khi control loop gửi vector trạng thái đến RL serving. Serving trả về hành động được chọn bởi model production và có thể trả thêm hành động của model staging để so sánh shadow mode. Control loop tính reward, cập nhật Prometheus metric và gọi hàm apply action để áp dụng hành động về ONOS.

Luồng MLOps gồm quá trình train DQN/PPO, log metric và artifact vào MLflow, lưu artifact vào MinIO, đăng ký model vào Model Registry, sau đó serving load model từ registry hoặc fallback sang local model nếu MLflow chưa sẵn sàng.

3.2.4. Nguyên lý hoạt động của hệ thống

Hệ thống vận hành theo một vòng lặp khép kín, kết nối chặt chẽ giữa hạ tầng mạng SDN, mô hình trí tuệ nhân tạo (RL) và quy trình tự động hóa MLOps:

Bước 1: Thu thập trạng thái: Mininet sinh lưu lượng mạng (bao gồm cả lưu lượng bình thường và tấn công). Thông qua REST API của ONOS Controller, State Collector trích xuất thông tin và xây dựng vector trạng thái 8 chiều (bao gồm: packet_rate, byte_rate, flow_count, flow_growth_rate, src_ip_entropy, latency, packet_loss, controller_cpu).

Bước 2: Phân tích & Ra quyết định: Control Loop gửi vector trạng thái này đến RL Serving (chạy thuật toán DQN hoặc PPO). Mô hình Production sẽ quyết định 1 trong 5 hành động để áp dụng (trong khi mô hình Staging có thể chạy ngầm để đối chiếu):

Action ID	Hành động	Ý nghĩa
0	no_action	Không can thiệp khi mạng bình thường
1	block_suspicious_flow	Chặn luồng đáng ngờ bằng flow rule.

2	limit_bandwidth	Giới hạn băng thông bằng meter/rate limit.
3	redirect_traffic	Chuyển hướng lưu lượng về honeypot.
4	isolate_device	Cô lập thiết bị nghi ngờ tấn công.

Bước 3: Đánh giá & Giám sát: Sau khi áp dụng hành động xuống ONOS, Control Loop tiến hành tính toán điểm thưởng (reward) dựa trên trạng thái mạng. Các chỉ số mạng được xuất sang Prometheus và trực quan hóa trên Grafana.

Bước 4: Tự động hóa MLOps: Nếu phát hiện điểm thưởng giảm dưới ngưỡng an toàn, hoặc mô hình Staging cho kết quả tốt hơn, Prometheus sẽ báo động cho AlertManager để gọi Webhook. Auto-trigger tiếp nhận webhook và tự động tạo Kubernetes Job để huấn luyện lại (retrain) hoặc thăng cấp (promote) mô hình thông qua MLflow Registry.

Luồng dữ liệu tổng thể: Mininet Traffic → ONOS Controller → State Collector → State Builder → Control Loop → RL Serving (DQN/PPO) → Action Decision → ONOS Flow Rule → Prometheus Metrics → Grafana Dashboard → AlertManager → Auto-trigger → Retrain/Promote → MLflow & MinIO.

3.3. Thiết kế môi trường huấn luyện

3.3.1. Môi trường offline

Huấn luyện dựa trên file CSV với vector trạng thái 8 chiều (packet rate, byte rate, flow count, flow growth rate, src IP entropy, latency, packet loss, controller CPU). Hàm thưởng (reward) được tính toán dựa trên độ chính xác của hành động và các hình phạt (penalty) về QoS.

Vector trạng thái gồm 8 đặc trưng:

```
{ packet_rate,
  byte_rate,
  flow_count,
  flow_growth_rate,
  src_ip_entropy,
  latency,
  packet_loss,
  controller_cpu
}
```

3.3.2. Môi trường Online với ONOS Controller

Trích xuất dữ liệu trực tiếp thông qua REST API của ONOS Controller để tính toán các đặc trưng (features) theo thời gian thực.

Trong repo, control_loop/state_collector.py đang thực hiện việc gọi ONOS tại `http://controller:8181/onos/v1/flows`, sau đó tính packet rate, byte rate, flow count, flow growth rate và các chỉ số liên quan.

```
ONOS_URL = "http://controller:8181/onos/v1"
AUTH = ("onos", "rocks")

prev_packets = 0
prev_bytes = 0
prev_flow = 0
prev_time = time.time()

return {
    "packet_rate": packet_rate,
    "byte_rate": byte_rate,
    "flow_count": flow_count,
    "flow_growth_rate": flow_growth_rate,
    "src_ip_entropy": entropy,
    "latency": latency,
    "packet_loss": packet_loss,
    "controller_cpu": controller_cpu,
}
```

3.4. Thiết kế pipeline MLOps end-to-end

- Data & Train: Thu thập và xử lý dữ liệu từ nhiều kịch bản tấn công (DDoS, Spoofing, Scanning). Tiến hành huấn luyện (DQN, PPO) qua nhiều seed và log kết quả tốt nhất vào MLflow.
- Evaluate & Deploy: Mô hình được đánh giá nghiêm ngặt (dựa trên mean reward, switching rate) trước khi lưu vào Registry. Khi triển khai, RL Serving tải mô hình từ Registry (DQN chạy port 8000, PPO port 8001).
- Monitor: Control Loop và RL Serving liên tục xuất metric (độ trễ, điểm thưởng, số lượng flow). Nếu AlertManager phát hiện suy giảm hiệu suất (vd: reward thấp), hệ thống sẽ tự động kích hoạt quá trình huấn luyện lại (retrain) hoặc đánh giá lại.

Chương IV. TRIỂN KHAI HỆ THỐNG

4.1. Container hóa các thành phần

4.1.1. Dockerfile và cấu hình image

Để đảm bảo khả năng tái lập môi trường và tự động hóa theo chuẩn cloud-native, hệ thống không chạy nguyên khối mà được phân tách thành các container độc lập theo từng chức năng:

- Đồng bộ môi trường: Danh sách các thư viện cốt lõi (xử lý dữ liệu, học sâu, dịch vụ web, MLOps, giám sát) được quản lý tập trung trong file requirements.txt, giúp Docker image cài đặt đồng bộ và tránh sai lệch khi triển khai.
- Tách biệt Runtime và Serving:
 - + Control Loop: Đóng vai trò là vòng lặp điều khiển trung tâm, kết nối mạng ONOS và gọi mô hình để thực thi hành động.
 - + RL Serving API: Hoạt động như một dịch vụ suy luận độc lập, cung cấp endpoint /predict cho mô hình DQN/PPO. Việc tách rời này giúp dễ dàng mở rộng và thay thế mô hình mà không làm gián đoạn toàn bộ pipeline.
- Tích hợp giám sát: Module metrics.py mở cổng cho phép Prometheus thu thập dữ liệu thời gian thực (reward, độ trễ, số lượng flow), từ đó làm cơ sở cho Grafana trực quan hóa.
- Tự động hóa MLOps: Hệ thống có một dịch vụ webhook in-cluster nhận cảnh báo từ Alertmanager.
 - + Nếu nhận cảnh báo mô hình kém (Low_Reward_Detected), hệ thống tự động tạo Kubernetes Job để huấn luyện lại.
 - + Nếu nhận cảnh báo mô hình Staging hoạt động tốt hơn, hệ thống sẽ gọi MLflow để thăng cấp (promote) mô hình lên Production và tự động reload lại RL Serving.
- Các image triển khai: Hệ thống sử dụng kết hợp các image có sẵn (ONOS 2.7.0, MLflow v3.0.0, MinIO, Prometheus, Grafana, AlertManager) và các Dockerfile tự xây dựng cho Control Loop, RL Serving, Auto Trigger (với cấu hình PYTHONPATH=/app).

Thành phần	Docker image/Dockerfile
ONOS	onosproject/onos:2.7.0

MLflow	ghcr.io/mlflow/mlflow:v3.0.0
MinIO	minio/minio:latest
Prometheus	prom/prometheus:latest
Grafana	grafana/grafana:latest
AlertManager	prom/alertmanager:latest
Control Loop	control_loop/Dockerfile
RL Serving	Dockerfile serving cho DQN/PPO
Auto Trigger	mlops/Dockerfile

4.1.2. Triển khai trên Kubernetes/K3s

Sau khi các thành phần chính của hệ thống được container hóa, hệ thống được triển khai trên môi trường Kubernetes/K3s thay vì sử dụng Docker Compose. Kubernetes được lựa chọn vì phù hợp với yêu cầu vận hành theo hướng cloud-native, hỗ trợ quản lý vòng đời container, tự động khởi động lại khi lỗi, mở rộng service, quản lý cấu hình và secrets, đồng thời phù hợp với định hướng của đề tài.

Trong hệ thống, các thành phần được triển khai trong namespace riêng là *sdn-security*. Việc sử dụng namespace giúp cô lập tài nguyên của hệ thống SDN Security với các workload khác trong cluster, đồng thời giúp việc quản lý, kiểm tra log và xóa tài nguyên thuận tiện hơn.

Các nhóm tài nguyên Kubernetes chính gồm:

Nhóm tài nguyên	Vai trò
Namespace	Tạo không gian triển khai riêng cho hệ thống
Deployment	Triển khai các service như ONOS, MLflow, RL Serving, Control Loop, Grafana, Prometheus, AlertManager và Auto-trigger.
StatefulSet	Triển khai MinIO để lưu trữ artifact ổn định hơn.
Service	Cung cấp endpoint nội bộ hoặc NodePort để truy cập các thành phần.
ConfigMap	Lưu cấu hình Prometheus, AlertManager và Grafana dashboard
Secret	Lưu thông tin truy cập MinIO như access key và secret key.

CronJob	Tự động chạy lại pipeline huấn luyện DQN/PPO theo lịch.
Job	Tạo bucket MinIO hoặc chạy các tác vụ khởi tạo.
HPA	Tự động scale RL Serving dựa trên tải nguyên CPU.

Các file Kubernetes chính của hệ thống gồm:

File	Chức năng
base.yaml	Khai báo namespace, service account, RBAC, secrets, ConfigMap, Service, Prometheus, Grafana và AlertManager config.
serving.yaml	Triển khai MinIO, MLflow, ONOS Controller, RL Serving DQN/PPO, Control Loop, Dashboard và Auto-trigger.
monitoring-jobs.yaml	Triển khai Prometheus, Grafana, AlertManager và các CronJob huấn luyện DQN/PPO.

Quy trình triển khai hệ thống trên Kubernetes/K3s được thực hiện như sau:

```
kubectl apply -f k8s/base.yaml
kubectl apply -f k8s/serving.yaml
kubectl apply -f k8s/monitoring-jobs.yaml
```

Sau khi triển khai, có thể kiểm tra trạng thái các pod và service bằng các lệnh:

```
kubectl get pods -n sdn-security -o wide
```

```
q23520964@vm-nt113:~/autonomous_sdn_security$ sudo kubectl get pods -n sdn-security -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
alertmanager-78cc5c477d-2cnnv	1/1	Running	0	29h	10.42.0.56	vm-nt113	<none>	<none>
auto-trigger-5dfdb66c9-d475s	1/1	Running	0	17h	10.42.0.89	vm-nt113	<none>	<none>
control-loop-84756f9bc-kkl8f	1/1	Running	0	20h	10.42.0.81	vm-nt113	<none>	<none>
controller-5f887989f4-1x2k9	1/1	Running	0	34h	10.148.0.2	vm-nt113	<none>	<none>
dqn-training-job-29667000-wdssx	1/1	Running	0	2m41s	10.42.0.93	vm-nt113	<none>	<none>
grafana-5c668ddd9f-fgqrp	1/1	Running	0	26h	10.42.0.77	vm-nt113	<none>	<none>
mlflow-c5b6d4574-kpzms	1/1	Running	1 (27h ago)	29h	10.42.0.53	vm-nt113	<none>	<none>
prometheus-6496797484-hkcfk	1/1	Running	0	27h	10.42.0.72	vm-nt113	<none>	<none>
retrain-dqn-1779955666-hwbvz	1/1	Running	0	17h	10.42.0.90	vm-nt113	<none>	<none>
retrain-dqn-1779965327-fh6hr	1/1	Running	0	15h	10.42.0.91	vm-nt113	<none>	<none>
retrain-dqn-1779967239-2crjv	1/1	Running	0	14h	10.42.0.92	vm-nt113	<none>	<none>
rl-serving-dqn-c968d89b5-5hgxx	1/1	Running	0	18h	10.42.0.84	vm-nt113	<none>	<none>
rl-serving-ppo-558dc48666-snh9m	1/1	Running	0	18h	10.42.0.85	vm-nt113	<none>	<none>
s3-0	1/1	Running	0	34h	10.42.0.40	vm-nt113	<none>	<none>
s3-setup-job-z8t45	0/1	Completed	0	34h	10.42.0.42	vm-nt113	<none>	<none>
sdn-dashboard-85bdc5cb6d-t4ngm	1/1	Running	0	34h	10.42.0.47	vm-nt113	<none>	<none>
sdn-defense-agent-748bbfc48b-2cgg2	1/1	Running	0	18h	10.42.0.83	vm-nt113	<none>	<none>

Hình. Kiểm tra các Pod trong namespace

```
kubectl get svc -n sdn-security
```

```
q23520964@vm-nt113:~/autonomous_sdn_security$ sudo kubectl get svc -n sdn-security -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
alertmanager	NodePort	10.43.60.74	<none>	9093:30093/TCP	34h	app=alertmanager
auto-trigger	ClusterIP	10.43.7.155	<none>	5001/TCP	34h	app=auto-trigger
control-loop	ClusterIP	10.43.45.167	<none>	9100/TCP	34h	app=control-loop
controller	NodePort	10.43.191.75	<none>	8181:30081/TCP	34h	app=controller
grafana	NodePort	10.43.59.162	<none>	3000:30000/TCP	34h	app=grafana
mlflow	NodePort	10.43.69.158	<none>	5000:30007/TCP	34h	app=mlflow
prometheus	NodePort	10.43.139.113	<none>	9090:30090/TCP	34h	app=prometheus
rl-serving-dqn	NodePort	10.43.103.209	<none>	8000:31061/TCP,9002:30002/TCP	34h	app=rl-serving-dqn
rl-serving-ppo	NodePort	10.43.76.233	<none>	8001:30377/TCP,9003:30003/TCP	34h	app=rl-serving-ppo
s3	NodePort	10.43.56.138	<none>	9005:30005/TCP,9001:30001/TCP	34h	app=s3
sdn-dashboard	NodePort	10.43.109.176	<none>	8501:30501/TCP	34h	app=sdn-dashboard
sdn-defense-agent	ClusterIP	10.43.109.23	<none>	8000/TCP	34h	app=sdn-defense-agent

Hình. Kiểm tra các service

```
kubectl get deployments -n sdn-security
```

```
g23520964@vm-nt113:~/autonomous_sdn_security$ sudo kubectl get deployment -n sdn-security -o wide
NAME          READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES                               SELECTOR
alertmanager   1/1     1             1           29h   alertmanager prom/alertmanager:latest         app=alertmanager
auto-trigger   1/1     1             1           34h   trigger      hmm0411/sdn-auto-trigger:latest     app=auto-trigger
control-loop   1/1     1             1           34h   loop         hmm0411/sdn-control-loop:latest     app=control-loop
controller     1/1     1             1           34h   onos         onosproject/onos:2.7.0              app=controller
grafana        1/1     1             1           29h   grafana      grafana/grafana:latest              app=grafana
mlflow         1/1     1             1           34h   mlflow       ghcr.io/mlflow/mlflow:v3.0.0        app=mlflow
prometheus     1/1     1             1           29h   prometheus   prom/prometheus:latest              app=prometheus
rl-serving-dqn 1/1     1             1           34h   serving      hmm0411/sdn-rl-serving:latest        app=rl-serving-dqn
rl-serving-ppo 1/1     1             1           34h   serving      hmm0411/sdn-rl-serving:latest        app=rl-serving-ppo
sdn-dashboard  1/1     1             1           34h   dashboard    python:3.10-slim                     app=sdn-dashboard
sdn-defense-agent 1/1     1             1           34h   agent        python:3.10-slim                     app=sdn-defense-agent
```

Hình. Danh sách các Deployment trong namespace

```
kubectl get cronjob -n sdn-security
```

```
g23520964@vm-nt113:~/autonomous_sdn_security$ sudo kubectl get cronjob -n sdn-security -o wide
NAME                SCHEDULE   TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE   CONTAINERS   IMAGES                               SELECTOR
dqn-training-job    0 2 * * *   <none>     False     1         3m7s           29h   trainer      hmm0411/sdn-rl-agent:latest         <none>
ppo-training-job    30 3 * * *   <none>     False     0         22h            29h   trainer      hmm0411/sdn-rl-agent:latest         <none>
```

Hình. Danh sách các CronJob trong namespace

```
kubectl get job -n sdn-security
```

```
g23520964@vm-nt113:~/autonomous_sdn_security$ sudo kubectl get job -n sdn-security -o wide
NAME                STATUS    COMPLETIONS   DURATION   AGE   CONTAINERS   IMAGES                               SELECTOR
dqn-training-job-29667000 Running  0/1           3m14s     3m14s   trainer      hmm0411/sdn-rl-agent:latest         batch.kubernetes.io/controller-uid=dc4869e-d771-4c13-8fa2-0813907abcb
retrain-dqn-177995666 Running  0/1           17h       17h     trainer      hmm0411/sdn-rl-agent:latest         batch.kubernetes.io/controller-uid=d3b67272-53f6-4dd9-8c82-d4431a0b0ca5
retrain-dqn-177996327 Running  0/1           15h       15h     trainer      hmm0411/sdn-rl-agent:latest         batch.kubernetes.io/controller-uid=36ade63a-8c9f-44bc-8da9-b993bc9bbdb
retrain-dqn-1779967239 Running  0/1           14h       14h     trainer      hmm0411/sdn-rl-agent:latest         batch.kubernetes.io/controller-uid=800df5fa-49ac-494b-bdf1-cba24fc3774
s3-setup-job        Complete 1/1           21s       34h     s3-setup     minio/minio:latest                 batch.kubernetes.io/controller-uid=ece53de2-7b5f-4238-bd3f-bc379d75bbdd
```

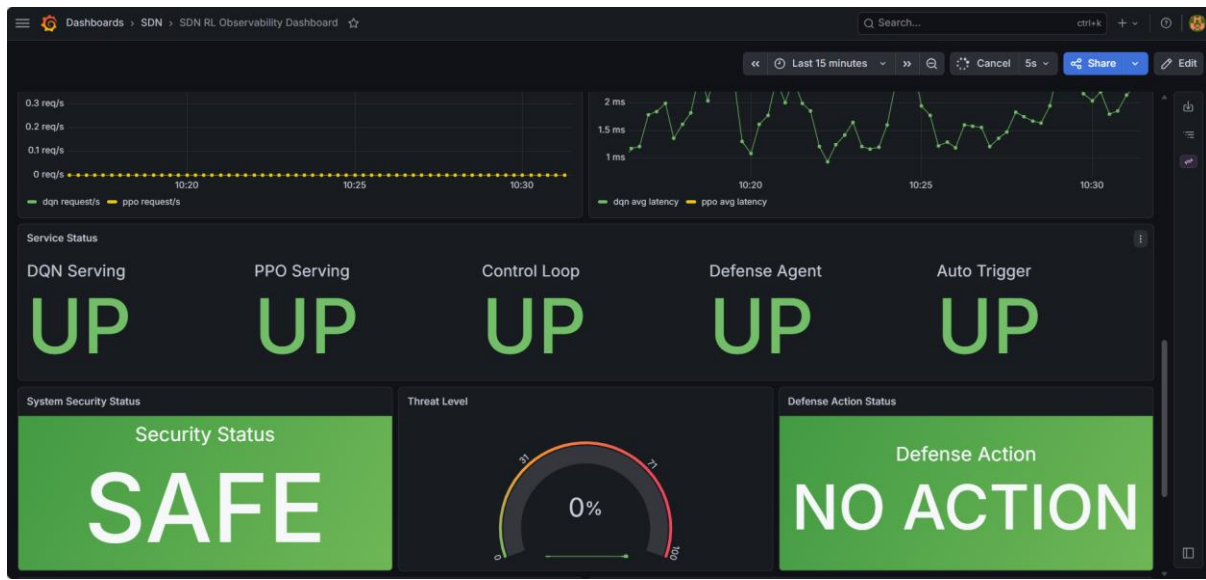
Hình. Danh sách các Job huấn luyện và huấn luyện lại mô hình trong namespace

Các service quan trọng được expose bằng Service để các thành phần có thể giao tiếp với nhau trong cluster. Ví dụ, Control Loop gọi RL Serving thông qua địa chỉ nội bộ rl-serving-dqn:8000 hoặc rl-serving-ppo:8001, Prometheus scrape metrics từ các service tương ứng, AlertManager gửi webhook đến auto-trigger:5001, còn MLflow sử dụng MinIO làm nơi lưu artifact.

Một số service cần truy cập từ bên ngoài được cấu hình dạng NodePort, chẳng hạn:

Service	Port nội bộ	NodePort	Mục đích
Grafana	3000	30000	Truy cập dashboard giám sát
MLflow	5000	30007	Theo dõi experiment và model registry
Prometheus	9090	30090	Kiểm tra metrics và alert rules
AlertManager	9093	30093	Theo dõi alert
MinIO API	9005	30005	Lưu artifact MLflow
MinIO Console	9001	30001	Quản trị MinIO

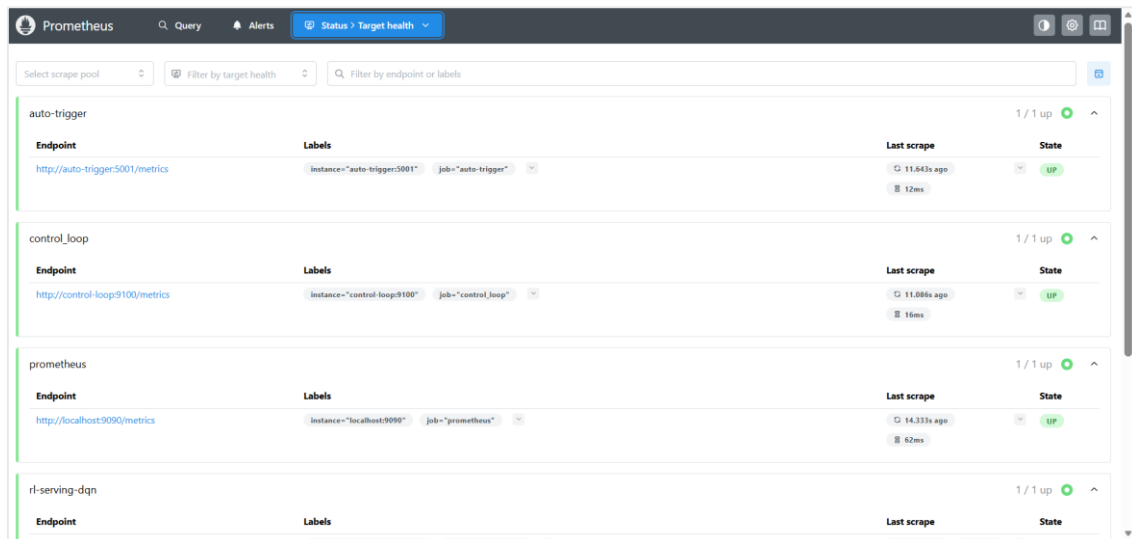
Nhờ triển khai trên Kubernetes/K3s, hệ thống có thể vận hành theo hướng gần với môi trường production hơn so với Docker Compose. Các service có thể được quản lý tập trung, restart tự động khi lỗi, cấu hình bằng ConfigMap/Secret, mở rộng bằng HPA và tự động huấn luyện lại bằng CronJob. Đây là nền tảng phù hợp để triển khai pipeline MLOps end-to-end cho hệ thống bảo mật mạng SDN tự động dựa trên học máy tăng cường.



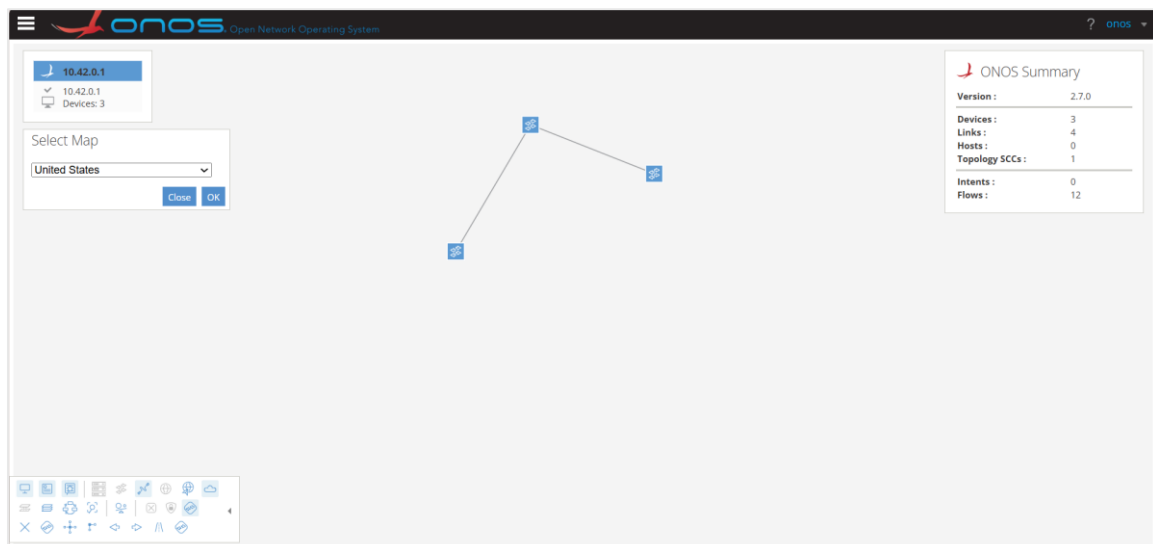
Hình. Dashboard giám sát hệ thống SDN Security trên Grafana

Run Name	Created	Dataset	Duration	Source	Models
PPO_Production_Trai...	1 minute ago	-	-	train_pp...	-
DQN_Production_Trai...	1 hour ago	-	-	train_dq...	-
DQN_Production_Trai...	16 hours ago	-	-	train_dq...	-
DQN_Production_Trai...	16 hours ago	-	-	train_dq...	-
DQN_Production_Trai...	19 hours ago	-	-	train_dq...	-
PPO_Production_Trai...	1 day ago	-	2.6min	train_pp...	model
DQN_Production_Trai...	1 day ago	-	4.9min	train_dq...	SDN_DQN_Model v6 +1
DQN_Production_Trai...	1 day ago	-	5.0min	train_dq...	model
DQN_Production_Trai...	1 day ago	-	4.8min	train_dq...	model
DQN_Production_Trai...	1 day ago	-	8.0min	train_dq...	model
DQN_Production_Trai...	1 day ago	-	8.0min	train_dq...	model
PPO_Production_Trai...	1 day ago	-	6.7min	train_pp...	model
DQN_Production_Trai...	1 day ago	-	7.1min	train_dq...	-
DQN_Production_Trai...	1 day ago	-	7.1min	train_dq...	-
DQN_Production_Trai...	1 day ago	-	5.0min	train_dq...	model

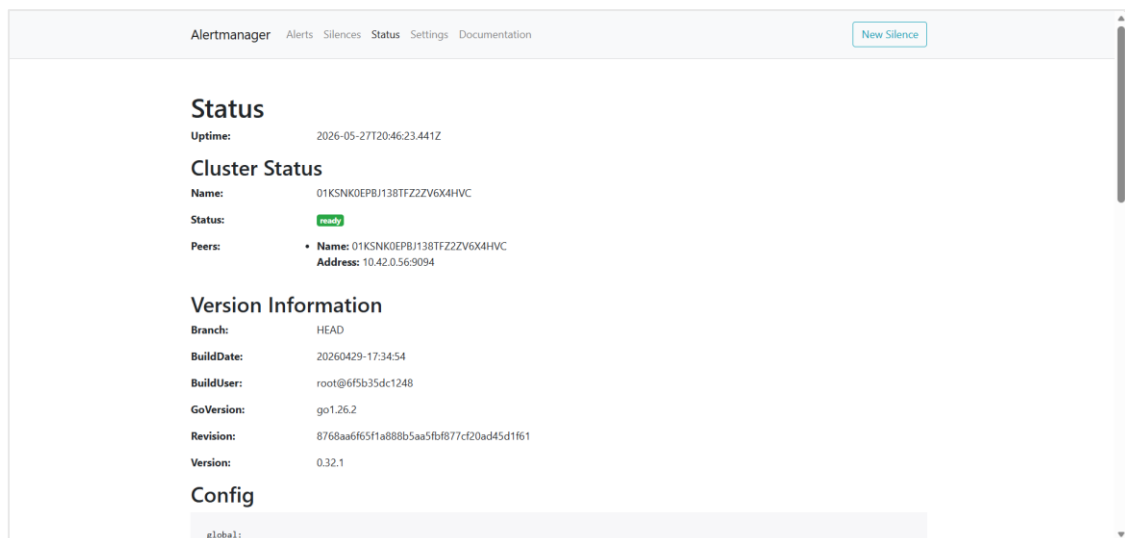
Hình. Giao diện quản lý thí nghiệm và phiên bản mô hình trên MLflow



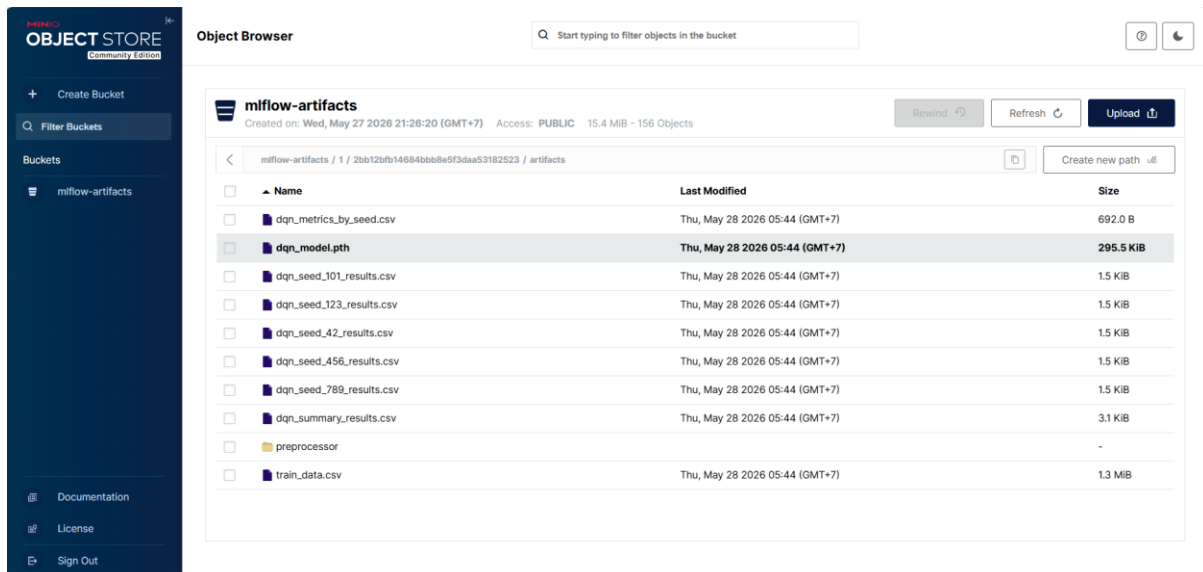
Hình. Trạng thái thu thập metric của Prometheus



Hình. Topology mạng được quản lý bởi ONOS Controller



Hình. Trạng thái hoạt động của Alertmanager



Hình. Kho lưu trữ artifact của MLflow trên MinIO

Chạy kiểm tra API predict:

- Đối với DQN:

```
curl -X POST http://127.0.0.1:8000/predict \
-H "Content-Type: application/json" \
-d '{
  "state": [0.1, 0.2, 10, 0.05, 0.3, 5, 0.01, 20]
}'
```

```
g23520964@vm-nt113:~/autonomous_sdn_security$ curl -X POST http://127.0.0.1:8000/predict \
-H "Content-Type: application/json" \
-d '{
  "state": [0.1, 0.2, 10, 0.05, 0.3, 5, 0.01, 20]
}'
{"action":0,"action_staging":0,"model":"DQN"}
```

Hình. Kiểm thử API suy luận của mô hình DQN

- Đối với PPO

```
curl -X POST http://127.0.0.1:8001/predict \
-H "Content-Type: application/json" \
-d '{
  "state": [0.1, 0.2, 10, 0.05, 0.3, 5, 0.01, 20]
}'
```

```
g23520964@vm-nt113:~/autonomous_sdn_security$ curl -X POST http://127.0.0.1:8001/predict \
-H "Content-Type: application/json" \
-d '{
  "state": [0.1, 0.2, 10, 0.05, 0.3, 5, 0.01, 20]
}'
{"action":0,"action_staging":0,"model":"PPO"}
```

Hình. Kiểm thử API suy luận của mô hình PPO

4.1.3. Quản lý biến môi trường và secrets

Đối với một hệ thống MLOps, nhóm giá trị nên quản lý bằng biến môi trường thường bao gồm: đường dẫn model, cấu hình lưu trữ artefact, endpoint của các dịch vụ nội bộ, cổng dịch vụ, và các tham số triển khai như chế độ chạy CPU/GPU hoặc chế độ test/production. Các thông tin nhạy cảm như MinIO access key và secret key được quản lý bằng Secret minio-credentials. Các deployment sử dụng biến môi trường như:

Biến môi trường	Ý nghĩa
MLFLOW_TRACKING_URI	Địa chỉ Mlflow Tracking Server
MLFLOW_S3_ENDPOINT_URL	Địa chỉ MinIO endpoint
AWS_ACCESS_KEY_ID	Access key để Mlflow ghi artifact
AWS_SECRET_ACCESS_KEY	Secret key để Mlflow ghi artifact
MODEL_TYPE	Xác định service đang chạy DQN hay PPO
PYTHONPATH	Đường dẫn import module Python

Từ góc độ an toàn vận hành, cách làm này cũng giúp hạn chế rủi ro lộ thông tin nhạy cảm trong repository. Các giá trị như khóa truy cập dịch vụ, thông tin tài khoản, chuỗi kết nối hoặc cấu hình theo môi trường không nên lưu cứng trong file mã nguồn. Thay vào đó, chúng nên được inject qua biến môi trường hoặc cơ chế secrets của nền tảng triển khai.

4.2. CI/CD Pipeline

4.2.1. Thiết kế pipeline build, test, deploy

Pipeline CI/CD của hệ thống được thiết kế qua ba giai đoạn chính (Build, Test, Deploy) nhằm phát hiện lỗi sớm và giảm rủi ro vận hành. Quá trình CI được tự động hóa hoàn toàn bằng GitHub Actions với các trọng tâm sau:

- Kiểm tra chất lượng & Kiểm thử: Pipeline thiết lập môi trường Python 3.10 và kiểm tra lỗi cú pháp bằng flake8. Tiếp theo, hệ thống thực hiện kiểm thử tích hợp giữa các phân hệ (rl_engine/, mlops/, v.v.) để đảm bảo các module RL và giám sát có thể khởi động đồng thời mà không gặp lỗi phụ thuộc.
- Đóng gói (Build & Push): Kế thừa từ một Dockerfile chuẩn hóa, quá trình build sẽ tải base image (Python 3.10 slim), cài đặt thư viện từ requirements.txt, thiết lập biến môi trường và tạo image hoàn chỉnh để đẩy (push) lên Docker Hub.

- Triển khai (Deploy): Khi toàn bộ các bước kiểm thử (CI) thành công, hệ thống chuyển sang giai đoạn triển khai trên môi trường Kubernetes/K3s. Việc ứng dụng container hóa giúp tự động hóa quá trình đưa ứng dụng lên production, loại bỏ các thao tác cấu hình thủ công phức tạp.

4.2.2. Smoke test trong chế độ CI

Smoke test được tích hợp trong pipeline CI nhằm kiểm tra nhanh độ ổn định của các thành phần cốt lõi thuộc chuỗi RL-MLOps-Monitoring sau mỗi lần thay đổi mã nguồn. Quá trình kiểm thử bao gồm các bước chính:

- Kiểm tra chất lượng mã nguồn: Sử dụng flake8 để rà soát lỗi cú pháp và lọc sớm các lỗi code cơ bản trước khi build image.
- Xác thực logic RL: Kiểm tra StateBuilder để đảm bảo chuyển đổi đúng trạng thái mạng thành vector đầu vào, và kiểm tra hàm tính Reward bằng dữ liệu mô phỏng.
- Kiểm thử giao tiếp và Control Loop: Đảm bảo hệ thống xuất thành công các metric quan trọng (như rl_reward, rl_latency, rl_flow_count) ra cổng Prometheus và thực hiện thành công một bài test log dữ liệu cơ bản lên MLflow.

4.2.3. Tự động kiểm tra chất lượng model trước khi deploy

Thay vì triển khai (deploy) trực tiếp sau khi huấn luyện, hệ thống áp dụng cơ chế tự động hóa hoàn toàn nhằm hạn chế rủi ro suy giảm hiệu năng. Mô hình mới chỉ được đưa vào vận hành nếu chứng minh được chất lượng vượt trội hơn phiên bản hiện tại.

- Cơ chế đánh giá song song:
 - + Tại môi trường runtime, main_loop.py liên tục lấy trạng thái mạng và gọi RL Serving API để nhận quyết định từ cả hai mô hình Production (hiện tại) và Staging (thử nghiệm).
 - + Sau khi tính toán điểm thưởng (reward) cho từng action, hệ thống xuất các metric (như rl_reward, rl_latency, switching_rate, v.v.) qua Prometheus. Việc này giúp đánh giá sát sao độ ổn định và hiệu năng thực tế của mô hình.

Metric	Ý nghĩa
mean_reward	Reward trung bình của model.
std_reward	Độ ổn định của reward.
switching_rate	Tần suất đổi hành động.

action_by_attack	Hành động có phù hợp với từng loại tấn công không.
Latency	Độ trễ suy luận.

- Cảnh báo và Huấn luyện lại tự động:
 - + Khi phát hiện chất lượng mô hình Production suy giảm (ví dụ: kích hoạt cảnh báo Low_Reward_Detected), AlertManager sẽ gửi webhook tới dịch vụ Auto-Trigger.
 - + Hệ thống tự động khởi tạo một Kubernetes Job (create_namespaced_job) để tiến hành huấn luyện lại (retrain) mô hình trong môi trường độc lập, đảm bảo hoàn toàn không ảnh hưởng tới tiến trình runtime. Kết quả và artifact sau đó được lưu trữ tự động lên MLflow và MinIO.
- Promote và triển khai liên tục:
 - + Trong manager.py, nếu theo dõi thấy mô hình Staging đạt hiệu năng tốt hơn Production dựa trên các tiêu chí đánh giá, MLflow Model Registry sẽ tự động chuyển trạng thái (promote) mô hình này sang "Production".
 - + Ngay lập tức, hệ thống gọi endpoint /reload của RL Serving API (DQN/PPO) để tải mô hình mới vào bộ nhớ. Cơ chế này giúp cập nhật mô hình liên tục mà không gây ra bất kỳ gián đoạn (downtime) nào cho mạng SDN.

Sau khi retraining hoàn tất, kết quả được ghi nhận lên MLflow Tracking Server dưới dạng metric, parameter, artifact và model version. Artifact của mô hình được lưu trên MinIO, giúp hệ thống quản lý tập trung vòng đời model và hỗ trợ truy xuất, rollback hoặc triển khai lại khi cần.

Trong manager.py, hệ thống sử dụng MLflow Model Registry để quản lý trạng thái của các model version. Khi mô hình staging đạt hiệu năng tốt hơn production, hệ thống thực hiện promote model bằng cách chuyển stage sang Production:

```
mlflow_client.transition_model_version_stage(
    name=best_model.name,
    version=best_model.version,
    stage="Production"
)
```

Sau khi promotion hoàn tất, hệ thống tiếp tục gửi yêu cầu reload tới RL Serving API:

```
requests.post("http://rl-serving-dqn:8000/reload")
```

Cơ chế này cho phép mô hình mới được áp dụng ngay trong runtime mà không cần dừng toàn bộ hệ thống. Đây là một đặc điểm quan trọng của hệ thống MLOps hiện đại, giúp giảm downtime và hỗ trợ continuous deployment cho AI model.

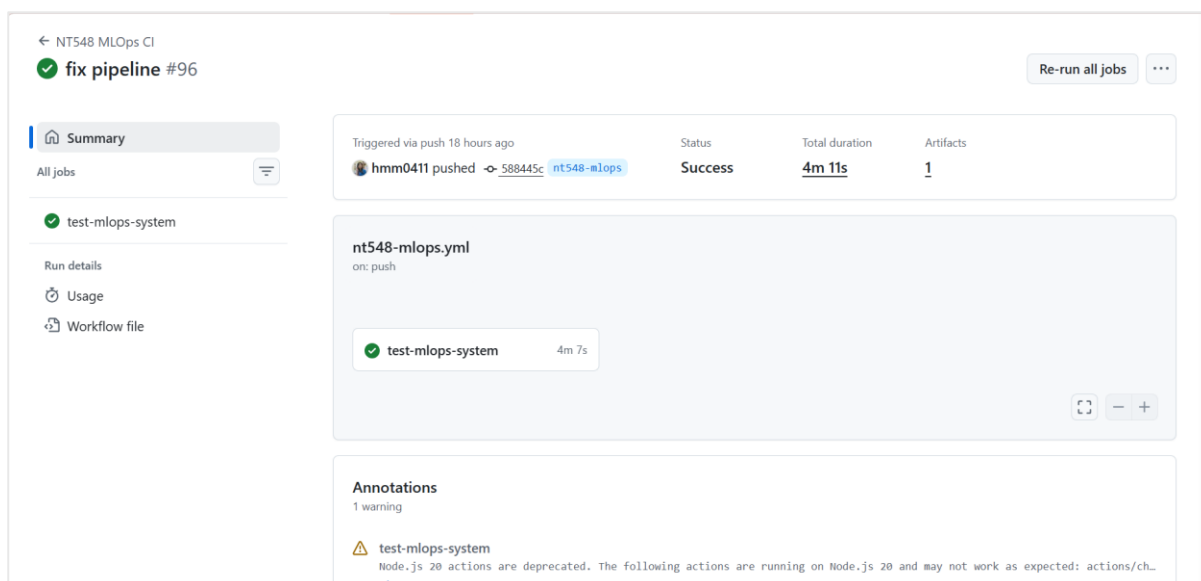
Ngoài ra, trong pipeline CI/CD, workflow nt549-ci.yml còn thực hiện kiểm tra nhanh quá trình training DQN/PPO và xác minh rằng các artifact huấn luyện được tạo thành công. Điều này đóng vai trò như một lớp validation ban đầu trước khi model được đưa vào runtime environment.

Nhìn tổng thể, cơ chế kiểm tra chất lượng model trong đồ án không chỉ dừng ở việc đánh giá offline trong giai đoạn training, mà được tích hợp trực tiếp vào runtime thông qua monitoring và alert automation. Toàn bộ quy trình được tổ chức theo chuỗi: Runtime Metrics → Prometheus → Alertmanager → Auto Trigger → Kubernetes Retrain Job → MLflow Tracking → Model Registry → Promote Production → Reload RL Serving API

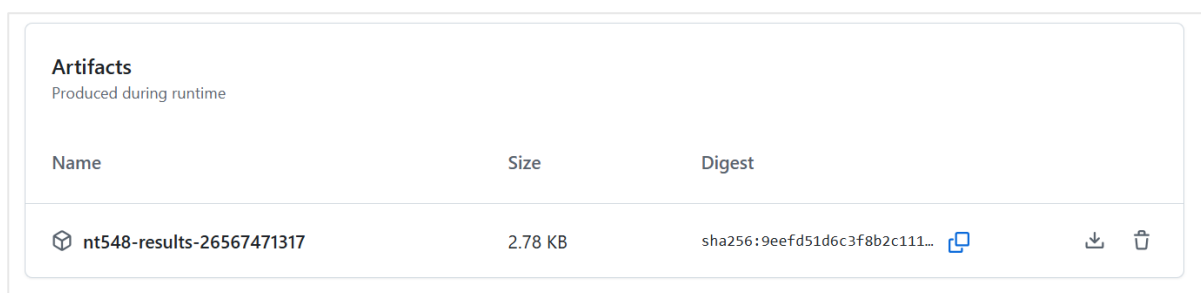
4.2.4. Github Actions CI/CD

Hệ thống sử dụng GitHub Actions để tự động hóa quy trình tích hợp liên tục (CI). Workflow chính nt548-ci.yml được kích hoạt mỗi khi có sự kiện push code lên nhánh phát triển:

- Giai đoạn validate: Kiểm tra cú pháp mã nguồn Python bằng flake8, xác thực định dạng các file cấu hình Kubernetes YAML và Dashboard Grafana JSON.
- Giai đoạn Test: Thực hiện các unit test và smoke test cho các module quan trọng như StateBuilder, Reward và giả lập gọi API suy luận để đảm bảo tính ổn định trước khi đóng gói.
- Hạn chế triển khai (CD): Workflow CD không thực hiện tự động triển khai trực tiếp từ GitHub lên cụm K3s. Điều này nhằm đảm bảo an toàn thông tin, tránh việc công khai cổng quản trị Kubernetes (6443) ra Internet, vốn dễ dẫn đến các nguy cơ tấn công chiếm quyền điều khiển.



Hình. Kết quả thực thi workflow CI trên GitHub Actions



Hình. Artifact được tạo bởi workflow CI

4.2.5. Triển khai CD nội bộ bằng deploy.sh

Để khắc phục hạn chế về bảo mật của GitHub-hosted runner, nhóm triển khai quy trình CD nội bộ (Local CD) thông qua script deploy.sh chạy trực tiếp trên máy ảo (VM):

- Kiểm tra môi trường: Script tự động kiểm tra kết nối tới cụm K3s (kubectl get nodes) và sự sẵn sàng của namespace sdn-security.
- Xác thực cấu hình: Thực hiện validate lại toàn bộ manifest và sự đồng bộ metric giữa mã nguồn với hệ thống giám sát trước khi áp dụng.
- Triển khai đồng bộ: Sử dụng lệnh kubectl apply cho các file base.yaml, serving.yaml và monitoring-jobs.yaml.
- Quản lý Rollout: Script theo dõi trạng thái của các Deployment (như control-loop, rl-serving, prometheus) và chỉ hoàn tất khi tất cả các Pod đạt trạng thái Successfully rolled out.

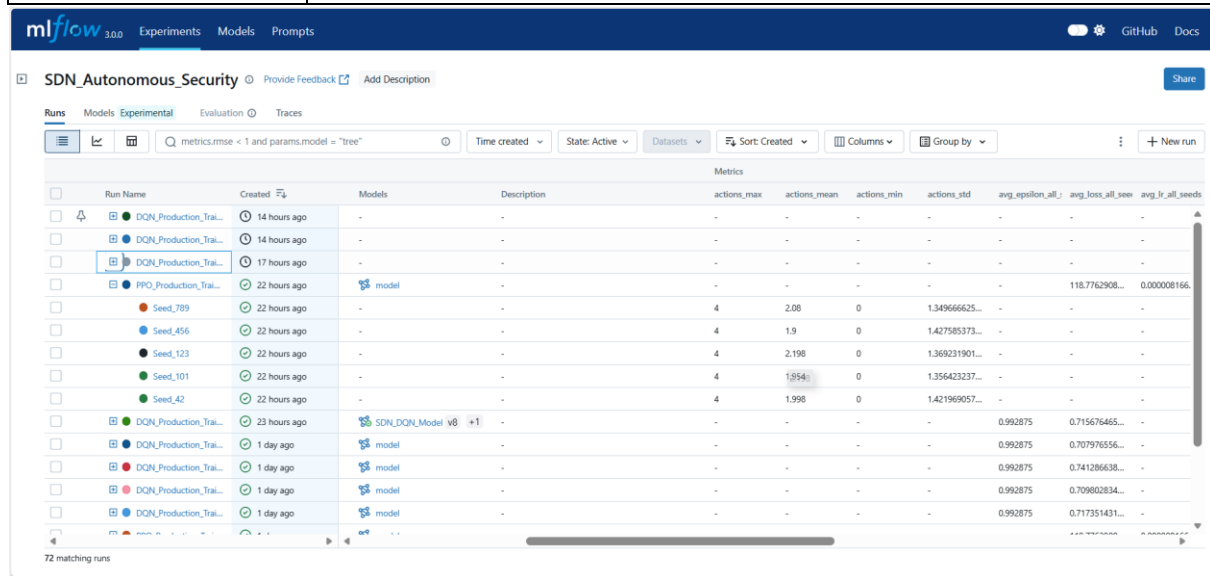
4.3. Quản lý vòng đời mô hình với Mlflow

Hệ thống tích hợp MLflow để quản lý tập trung và khoa học toàn bộ quy trình phát triển mô hình Reinforcement Learning.

4.3.1. Experiment Tracking

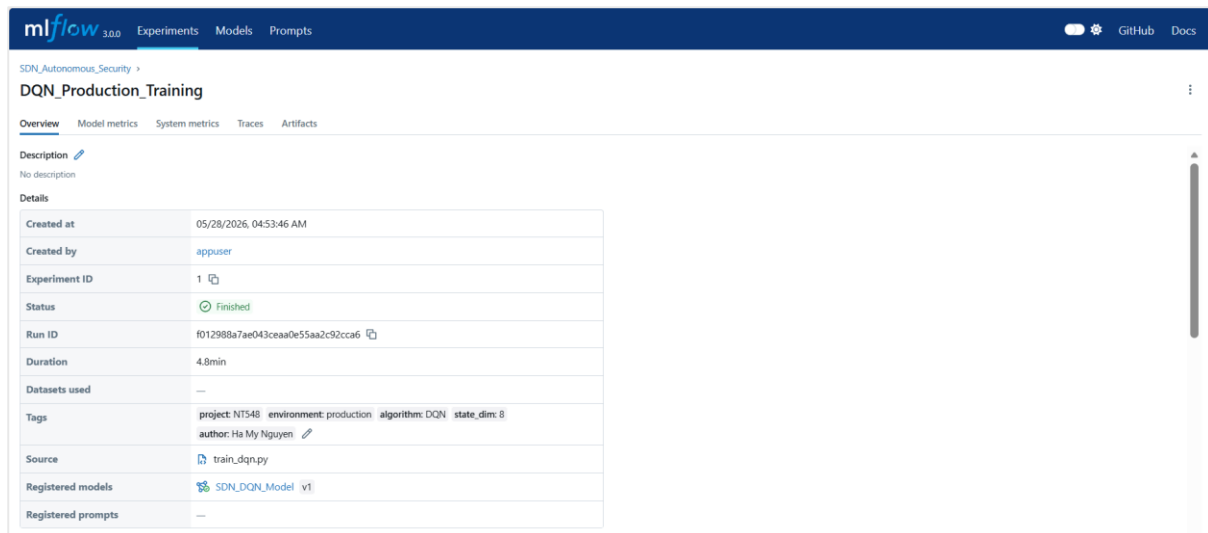
Trong quá trình huấn luyện, MLflow được sử dụng để log:

Nhóm thông tin	Các chỉ số chi tiết
Parameters	algorithm, state_dim, action_dim, gamma, learning rate, batch size, seeds.
Metrics	reward_raw, reward_mean, reward_std, reward_best, loss, epsilon.
Artifacts	model .pth, scaler .pkl, train data, summary CSV.
Tags	project, environment, algorithm, author.

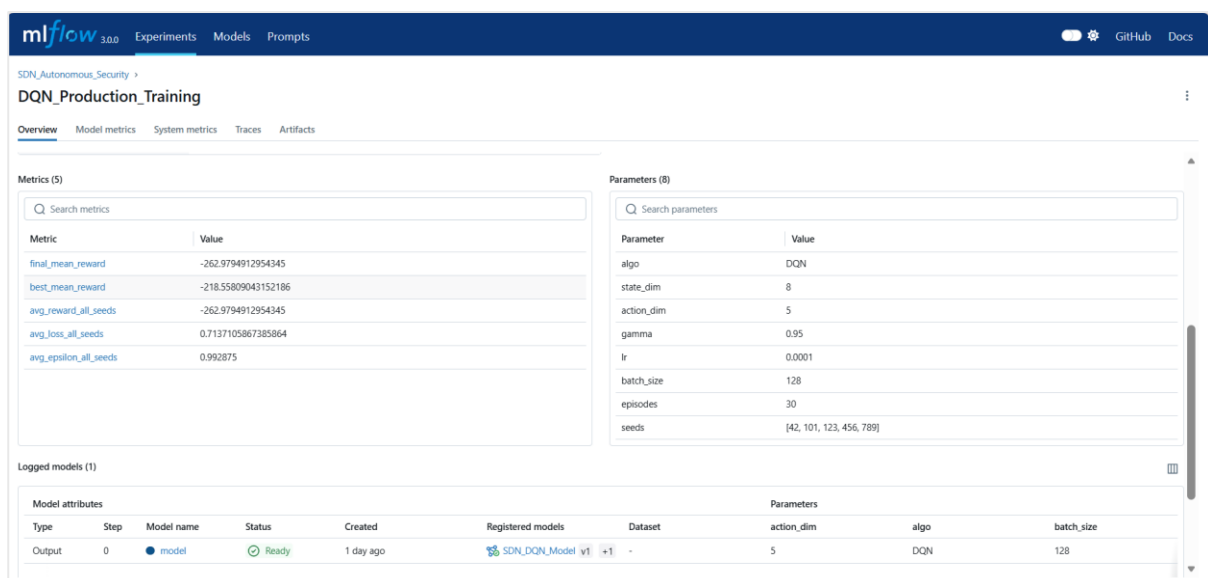


Hình. Danh sách các lần huấn luyện được quản lý bởi MLflow

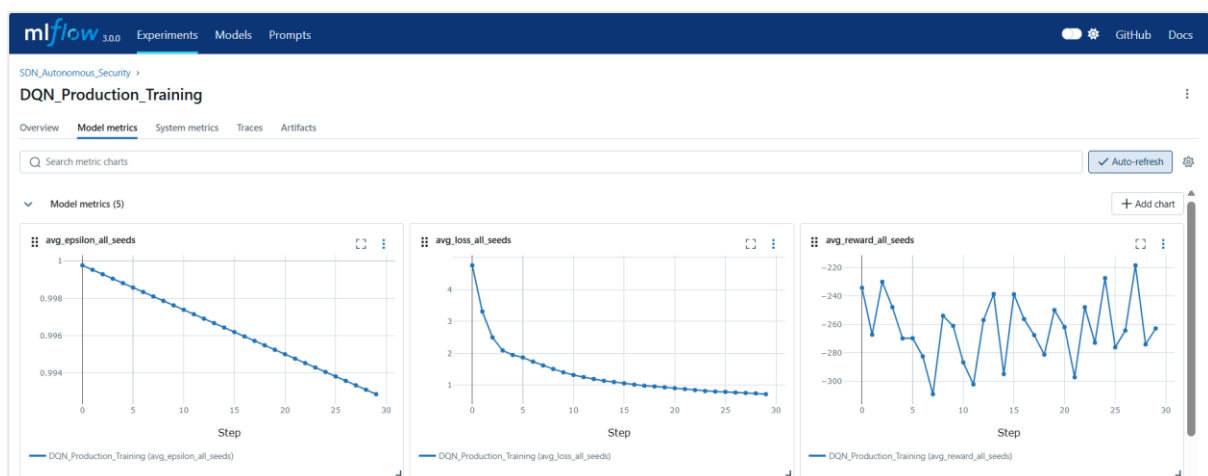
Các file train_dqn.py và train_ppo.py đã có logic log metric, artifact và model vào MLflow Registry.



Hình. Thông tin chi tiết của một lần huấn luyện DQN



Hình. Tham số và kết quả đánh giá của mô hình DQN trên MLflow



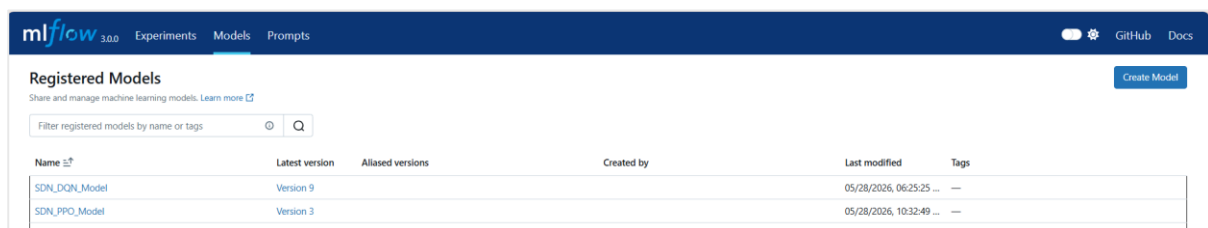
Hình. Biểu đồ theo dõi quá trình huấn luyện mô hình DQN

4.3.2. Model Registry

Model Registry quản lý version của mô hình. Trong đề tài, hai model chính là:

Model	Registry name
DQN	SDN_DQN_Model
PPO	SDN_PPO_Model

Mỗi model có thể có stage như Staging hoặc Production. RL serving ưu tiên load model ở stage Production. Nếu có model Staging, hệ thống có thể chạy song song để so sánh reward giữa staging và production.



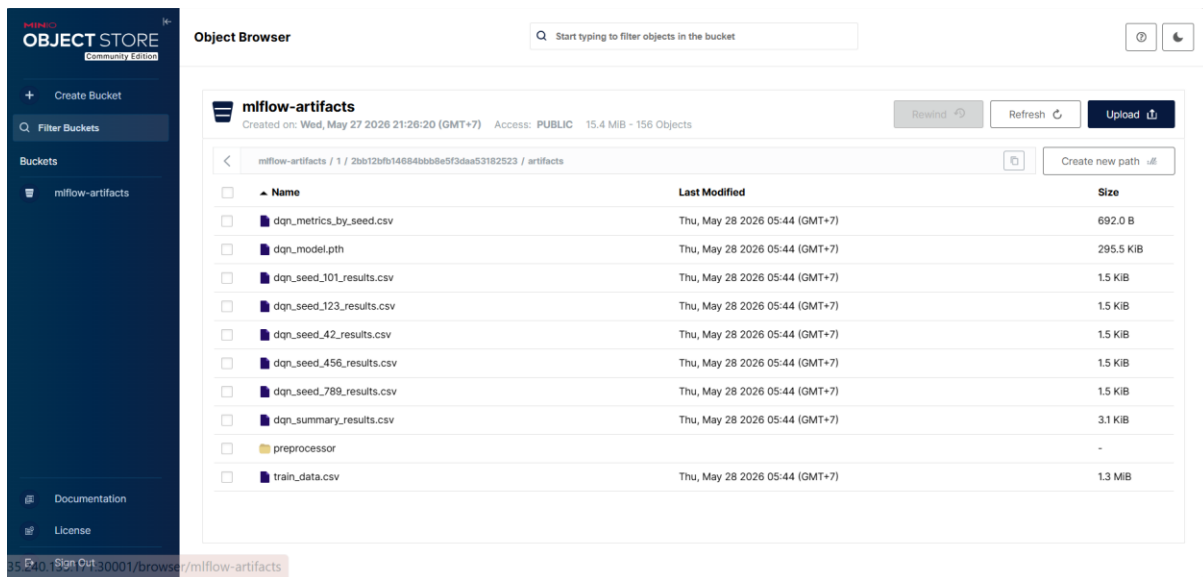
Hình. Danh sách mô hình được quản lý trên MLflow Model Registry

4.3.3. Artifact Storage với MinIO

Trong kiến trúc MLOps, việc tách biệt giữa quản lý Metadata (thông số, chỉ số) và Artifact (tệp tin vật lý) là cực kỳ quan trọng. Hệ thống sử dụng MinIO — một nền tảng lưu trữ đối tượng mã nguồn mở tương thích chuẩn S3 — để làm backend lưu trữ cho MLflow.

- Vai trò: MinIO lưu trữ toàn bộ các sản phẩm đầu ra của quá trình huấn luyện bao gồm tệp mô hình (.pth), bộ chuẩn hóa dữ liệu (scaler.pkl), các tập dữ liệu huấn luyện/kiểm thử và biểu đồ kết quả.
- Triển khai Cloud-native: Trong cụm Kubernetes, MinIO được vận hành dưới dạng StatefulSet để đảm bảo tính toàn vẹn của dữ liệu lưu trữ. Một Job khởi tạo sẽ tự động tạo bucket mlflow-artifacts ngay khi hệ thống được deploy.
- Kết nối: MLflow Tracking Server giao tiếp với MinIO thông qua các biến môi trường tiêu chuẩn: MLFLOW_S3_ENDPOINT_URL, AWS_ACCESS_KEY_ID và AWS_SECRET_ACCESS_KEY.
- Luồng lưu trữ:
 - + Huấn luyện DQN/PPO hoàn tất
 - + MLflow ghi nhận Metadata (params/metrics) vào cơ sở dữ liệu.

- + Tập tin mô hình và scaler được đẩy trực tiếp vào bucket trên MinIO.
- + Model Registry cập nhật phiên bản mới trở tới vị trí lưu trữ trên MinIO.



Hình. Bucket *mlflow-artifacts* trên MinIO dùng để lưu *model*, *scaler* và *artifact* của các lần huấn luyện.

4.3.4. Cơ chế load model Production/Staging/Fallback

Hệ thống RL Serving được thiết kế với cơ chế nạp mô hình thông minh, ưu tiên tính liên tục của dịch vụ (High Availability).

Thứ tự ưu tiên nạp mô hình:

- Production Stage: Service tự động kiểm tra và tải phiên bản mô hình được gắn nhãn "Production" từ MLflow Model Registry để phục vụ các yêu cầu thực tế.
- Staging Stage: Nếu tồn tại phiên bản "Staging", service sẽ tải song song để thực hiện đánh giá theo cơ chế Champion-Challenger (so sánh hiệu quả mà không làm ảnh hưởng tới luồng xử lý chính).
- Scaler: Hệ thống ưu tiên tải *scaler.pkl* tương ứng từ artifact trên MinIO để đảm bảo tính đồng bộ về tiền xử lý dữ liệu với mô hình.
- Cơ chế này giúp hệ thống vẫn có thể hoạt động trong trường hợp MLflow hoặc MinIO chưa sẵn sàng. Đây là điểm quan trọng trong vận hành production-like vì serving không nên dừng hoàn toàn chỉ vì registry tạm thời lỗi.

Cơ chế Fallback (Dự phòng):

- Trong trường hợp MLflow Server hoặc MinIO gặp sự cố (mất kết nối hoặc chưa có mô hình), hệ thống sẽ tự động kích hoạt chế độ Fallback.

- Service sẽ nạp các tệp mô hình lưu trữ cục bộ tại thư mục nội bộ: `models/dqn_model.pth` hoặc `models/ppo_model.pth`.

Việc thiết kế cơ chế fallback đảm bảo rằng hệ thống bảo mật SDN không bao giờ bị dừng hoàn toàn chỉ vì lỗi tại tầng quản lý Registry. Điều này duy trì khả năng phòng thủ mạng liên tục ngay cả trong điều kiện hạ tầng MLOps gặp sự cố tạm thời.

4.4. Hệ thống serving và control loop

4.4.1. Model serving endpoint

RL cung cấp các endpoint chính:

Endpoint	Chức năng
/health	Kiểm tra trạng thái service và model
/predict	Nhận vector state và trả về action
/reload	Reload model sau khi promote hoặc rollback

DQN serving chạy ở port 8000, PPO serving chạy ở port 8001. Metric Prometheus của DQN chạy ở 9002, PPO chạy ở 9003.

Kiểm tra trạng thái của các endpoint của mỗi thuật toán:

```
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8001/health
```

```
g23520964@vm-nt113:~/autonomous_sdn_security$ curl http://127.0.0.1:8000/health
{"has_production_model":false,"has_staging_model":false,"model_type":"dqn","status":"ok"}
g23520964@vm-nt113:~/autonomous_sdn_security$ curl http://127.0.0.1:8001/health
{"has_production_model":false,"has_staging_model":false,"model_type":"ppo","status":"ok"}
```

Hình. Kiểm tra trạng thái hoạt động của RL Serving API

4.4.2. Control loop tích hợp RL agent với ONOS Controller

Control loop thực hiện vòng lặp: lấy raw state từ ONOS → Build vector state 8 chiều → Gọi RL Serving → Nhận action production và staging → Tính reward → Apply action về ONOS → Export metric cho Prometheus
→ Lặp lại sau mỗi chu kỳ

```
while True:
    raw = get_state()
    state = np.array(state_builder.build(raw), dtype=np.float32)

    model_to_use = "dqn"

    action_prod, action_staging, model_name = get_action(state, model_type=model_to_use)
```

```

reward_prod = reward_calc.calculate(raw, action_prod)
reward_staging = reward_calc.calculate(raw, action_staging)

execute_action(action_prod)
update_metrics(state, reward_prod, reward_staging, model_name, action_prod)

print(f"[{model_name}] Prod Action={action_prod} (R={reward_prod}) | Staging
Action={action_staging} (R={reward_staging})")

time.sleep(SLEEP_TIME)

```

4.4.3. Chiến lược promote và rollback model

Chiến lược promote/rollback được thiết kế như sau:

Tình huống	Hành động
Model staging có reward cao hơn production	Promote staging lên production.
Production reward giảm dưới ngưỡng	Trigger retrain
Model mới gây reward thấp hoặc hành động bất ổn	Rollback về version trước
MLflow tạm thời không truy cập được	Serving fallback sang local model.

Trong auto-trigger, khi nhận alert Low_Reward_Detected hoặc PPO_Reward_Drop, hệ thống kích hoạt retrain. Khi nhận alert Staging_Outperforms_Production, hệ thống chạy evaluate, promote model và gọi /reload cho DQN/PPO serving.

4.5. Giám sát và tự động vận hành

4.5.1. Prometheus metrics

Các metric chính gồm:

Metric	Ý nghĩa
rl_reward	Reward runtime của production/staging
rl_latency	Độ trễ mạng hoặc độ trễ suy diễn liên quan.
rl_packet_loss	Tỷ lệ mất gói.
rl_flow_count	Số lượng flow đang hoạt động.

rl_action	Hành động model chọn.
rl_controller_cpu	CPU controller ước lượng/thu thập.
serving_requests_total	Số request gọi serving.
serving_latency_seconds	Độ trễ inference.
alerts_received_total	Số cảnh báo auto-trigger nhận được.

Hình. Trạng thái thu thập metric của Prometheus

4.5.2. Grafana dashboard

Grafana dashboard gồm các panel:

Panel	Nội dung
Reward Performance	So sánh reward production và staging
Flow Count	Theo dõi số flow trong mạng
Packet Loss	Theo dõi chất lượng mạng
Latency	Theo dõi độ trễ
RL Action History	Xem model chọn hành động nào theo thời gian
Serving Status	Kiểm tra DQN/PPO serving còn sống không
Alert Status	Theo dõi cảnh báo Prometheus/AlertManager



Hình. Dashboard giám sát hệ thống SDN Security trên Grafana

4.5.3. AlertManager

AlertManager nhận alert từ Prometheus và gửi đến auto-trigger. Các rule quan trọng gồm reward thấp, PPO reward drop, control loop anomaly, phát hiện tấn công và staging tốt hơn production.

4.5.4. Auto-trigger webhook

Auto-trigger là service Flask nhận webhook từ AlertManager. Khi có alert, service kiểm tra alertname và status. Nếu reward thấp, nó chạy data processor và retrain model. Nếu staging tốt hơn production, nó chạy evaluate, promote model và reload serving.

Chương V. THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Môi trường thực nghiệm

Môi trường thực nghiệm được triển khai theo hướng container hóa và cloud-native. Các thành phần gồm:

Thành phần	Công nghệ
Network simulator	Mininet
SDN Controller	ONOS 2.7.0
RL Model	DQN, PPO
Môi trường	Python 3.10
Container	Docker
Orchestration	K3s
Tracking	MLflow
Artifact Storage	MinIO
Monitoring	Prometheus
Visualization	Grafana
Alerting	AlertManager
CI/CD	Github Actions

5.2. Đánh giá pipeline CI/CD và MLOps

Pipeline CI/CD đã hỗ trợ kiểm tra lỗi cú pháp Python bằng flake8 và build Docker image cho control loop. Đây là bước quan trọng để đảm bảo source code không lỗi cơ bản trước khi đưa vào môi trường vận hành.

Pipeline MLOps đáp ứng các yêu cầu chính:

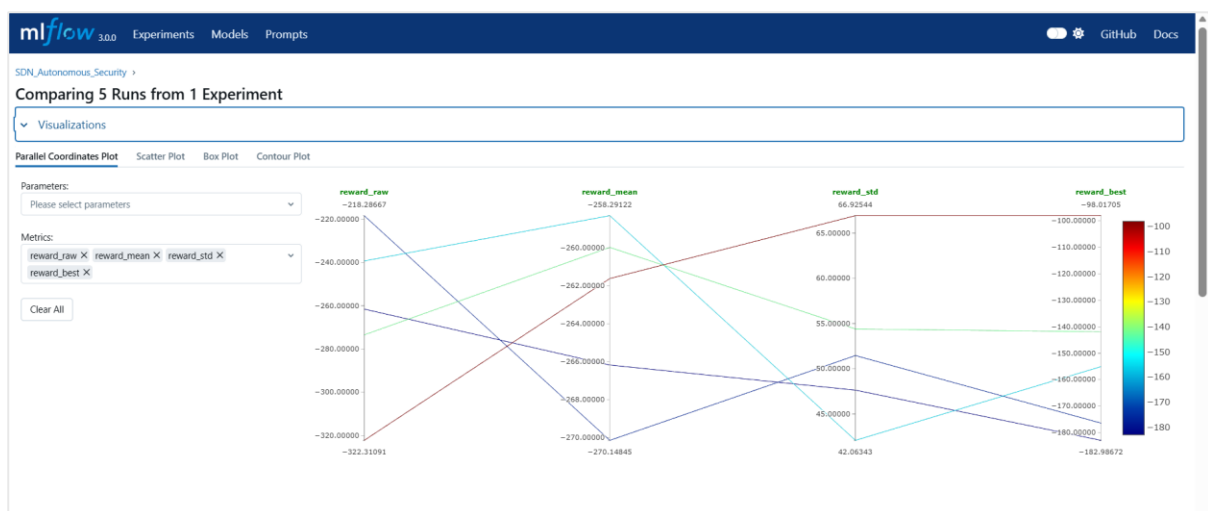
Yêu cầu	Đánh giá
Tracking thí nghiệm	Có log metric, parameter và artifact bằng MLflow.
Versioning model	Có Model Registry cho DQN/PPO.
Artifact storage	Có MinIO làm backend lưu artifact.
Serving model	Có API serving cho DQN/PPO.
Monitoring	Có Prometheus/Grafana.
Alerting	Có AlertManager.
Auto retrain/promote	Có auto-trigger webhook.

Rollback	Định hướng thông qua Model Registry và stage transition.
----------	--

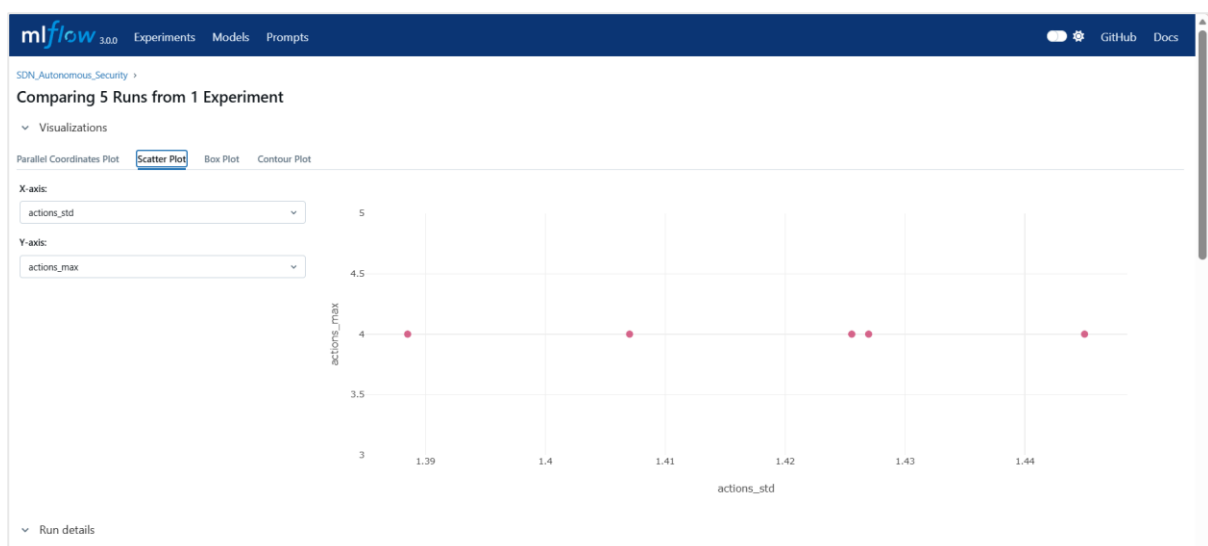
5.4. Đánh giá hệ thống giám sát

Hệ thống giám sát cho phép theo dõi trạng thái runtime của cả mô hình và mạng. Prometheus scrape các service, Grafana hiển thị dashboard, AlertManager phát cảnh báo khi metric vượt ngưỡng. Các metric quan trọng như reward, latency, packet loss, flow count và action giúp đánh giá mô hình không chỉ ở giai đoạn train mà còn trong quá trình vận hành.

- Đánh giá giữa các lần chạy với nhau trong Mlflow trong 1 experiment



Hình. So sánh kết quả huấn luyện giữa các lần chạy trên MLflow



Hình. Phân bố kết quả huấn luyện trên MLflow



Hình. Biểu đồ theo dõi các metric huấn luyện của mô hình PPO

SDN_Autonomous_Security >

Seed_789

Overview

Model metrics

System metrics

Traces

Artifacts

Registered prompts

Metrics (12)

Search metrics

Metric	Value
reward_raw	-190.67631617363347
reward_mean	-232.65039840523178
reward_std	55.08033861832408
reward_best	-114.7558544990952
loss_total	118.47299956870079
loss_policy	-4.0531160649948106e-9
loss_value	118.4729995727539
learning_rate	0.0000081666666666666673
actions_mean	2.08
actions_std	1.349666625504239
actions_min	0
actions_max	4

Hình. Tổng hợp các chỉ số đánh giá của một lần huấn luyện trên MLflow

Khi reward production giảm dưới ngưỡng, hệ thống có thể phát cảnh báo để kích hoạt retrain. Khi reward staging cao hơn production, hệ thống có thể kích hoạt promote model. Điều này thể hiện đặc trưng của MLOps: không chỉ deploy một lần mà còn liên tục giám sát và cải tiến model.

Qua quá trình thực nghiệm, hệ thống đã chứng minh được khả năng triển khai end-to-end pipeline MLOps cho bảo mật SDN. CI pipeline trên GitHub Actions kiểm tra thành công source code, dashboard, Kubernetes manifest và build Docker image. Local CD pipeline bằng deploy.sh triển khai thành công toàn bộ service lên K3s. Hệ thống Prometheus/Grafana hiển thị được metric runtime, Alertmanager gửi cảnh báo đến auto-trigger, và auto-trigger có thể tạo Kubernetes retrain job khi phát hiện reward thấp. Các lỗi thực tế như sai kubeconfig, Prometheus YAML, metric không đồng bộ, auto-trigger 401 Unauthorized và Grafana No Data đã được xử lý, giúp hệ thống ổn định hơn cho kịch bản demo cuối.

Chương VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Tổng kết kết quả đạt được

Đề tài đã xây dựng được kiến trúc MLOps cho hệ thống bảo mật SDN dựa trên học máy tăng cường. Hệ thống kết hợp ONOS Controller, State Collector, RL Agent, Control Loop, MLflow, MinIO, Prometheus, Grafana, AlertManager, Docker, Kubernetes và GitHub Actions nhằm hỗ trợ triển khai, giám sát và quản lý mô hình học máy trong môi trường SDN.

Nhóm đã Docker hóa các service và triển khai trên Kubernetes nhằm tăng khả năng quản lý và mở rộng hệ thống. Đồng thời, pipeline CI/CD kết hợp GitOps cũng được xây dựng để tự động hóa quá trình build và deploy ứng dụng.

Ngoài ra, hệ thống monitoring giúp theo dõi hiệu năng của hạ tầng và mô hình RL thông qua các metric như latency, reward và inference time. Kết quả đạt được cho thấy mô hình DevOps–MLOps có thể hỗ trợ vận hành hệ thống AI bảo mật SDN theo hướng tự động và ổn định hơn.

6.2. Hạn chế của hệ thống hiện tại

Hệ thống hiện tại chủ yếu được triển khai trong môi trường mô phỏng nên chưa đánh giá đầy đủ khả năng hoạt động ở quy mô lớn hoặc trong môi trường thực tế.

Mô hình **học máy tăng cường** mới tập trung vào một số trạng thái mạng và dạng tấn công cơ bản, chưa hỗ trợ nhiều kịch bản tấn công phức tạp. Ngoài ra, cơ chế retrain và rollback vẫn còn đơn giản và chưa tối ưu hoàn toàn cho production environment.

Bên cạnh đó, việc tối ưu hiệu năng Kubernetes cluster và khả năng chịu lỗi của hệ thống vẫn chưa được triển khai chuyên sâu.

6.3. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng trên môi trường SDN thực tế hoặc cloud infrastructure nhằm đánh giá khả năng hoạt động với lưu lượng lớn hơn.

Nhóm cũng định hướng nghiên cứu và tích hợp các thuật toán học máy tăng cường nâng cao nhằm cải thiện hiệu quả phát hiện và giảm thiểu tấn công mạng.

Ngoài ra, hệ thống monitoring và MLOps có thể tiếp tục được nâng cấp với cơ chế model drift detection và self-healing nhằm tăng khả năng tự động hóa và ổn định trong môi trường production.

NGUỒN THAM KHẢO

- [1] Andrew Chen, Andy Chow, Aaron Davidson, Arjun DCunha, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Clemens Mewald, Siddharth Murching, Tomas Nykodym, và các cộng sự. “Developments in mlflow: A system to accelerate the machine learning lifecycle”. In: Proceedings of the fourth international workshop on data management for end-to-end machine learning. 2020, pp. 1– 4.
- [2] Dominik Kreuzberger, Niklas K“uhl, và Sebastian Hirschl. “Machine Learn- ing Operations (MLOps): Overview”. In: Definition, and Architecture. arXiv 20222205 (2022).

LINK TÀI LIỆU

Link Drive tổng hợp:

https://drive.google.com/drive/folders/1Lw9J4-qOBOONHvEq4Ueurmmci3TB6hG?usp=drive_link

Link Mã nguồn:

https://github.com/hmm0411/autonomous_sdn_security/tree/nt548-mlops